

Tutorial - Introducción a Python y paquetes básicos para prácticas de Tratamiento de Señales Visuales (GITST)

Introducción a la Visión Artificial (GII)

Juan Carlos San Miguel Avedillo

*Escuela Politécnica Superior
Universidad Autónoma de Madrid*

Septiembre 2025

Contenidos

1	Instalación de software	2
1.1	Python	2
1.2	Visual Studio Code	2
1.3	Entorno de trabajo	3
2	Ejecutando código	5
2.1	Terminal	5
2.2	Visual Studio Code	6
3	Aspectos básicos	13
3.1	Sangría	13
3.2	Comentarios	13
3.3	Mostrar datos en el terminal	14
3.4	Variables	14
3.5	Condiciones	17
3.6	Bucles	17
3.7	Funciones	18
3.8	Clases	18
3.9	Uso de paquetes	19
3.10	Ficheros	20
4	Paquetes	21
4.1	Numpy	21
4.2	Matplotlib	25
4.3	SciPy	29
4.4	Scikit-image	32
5	Formato ficheros de las prácticas	35
6	Referencias	36

1 Instalación de software

El curso usa Python como lenguaje de programación y entornos virtuales para gestionar las librerías utilizadas por Python. En esta sección se explica como instalar el software necesario.

1.1 Python

En este curso se utiliza Python 3.12.0, que es la versión instalada en los ordenadores de los laboratorios. *Por lo tanto, versiones anteriores y/o posteriores de Python no serán compatibles con el material de clase. No se proporcionará soporte para versiones de Python distintas a 3.12.0.*

Puede verificar la versión que tiene instalada con el siguiente comando:

```
$ python -V
```

Observe que utilizaremos el símbolo \$ para las instrucciones que se deban ejecutar en el terminal de comandos.

No obstante, siéntase libre de usar una versión más actual de Python en su ordenador personal si tiene experiencia con Python. Eso sí, deberá verificar que su código ejecutado es compatible con Python 3.12.0

Si desea instalar Python en su ordenador personal, puede obtenerlo en <https://www.python.org/downloads>. En concreto, este tutorial utiliza la version 3.12.0 para Windows 64-bit que puede descargar en la página Moodle del curso y en el siguiente enlace <https://www.python.org/ftp/python/3.12.0/python-3.12.0-amd64.exe>. Descargue y ejecute el fichero para instalar correctamente Python 3.12.0 en Windows. En la primera ventana debe seleccionar la opción **Add Python.exe to PATH**. La Figura 1 muestra un ejemplo de las instalación exitosa de Python.

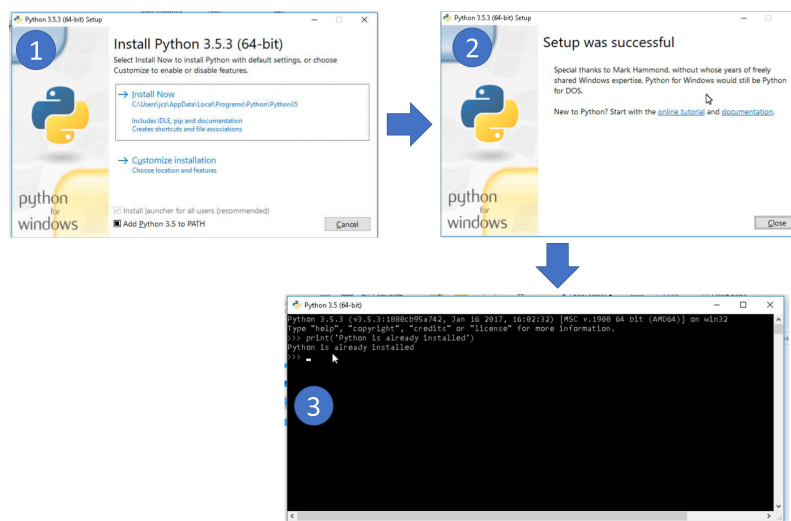


Fig. 1: Pasos de la instalación exitosa de Python 3.12.0 en Windows. Puede consultar todos los pasos en el vídeo disponible en <https://recordit.co/3wHUEwA5jA>. Aunque este vídeo es para la versión 3.5.3, es equivalente para la versión 3.12.0 disponible en Moodle

1.2 Visual Studio Code

Como entorno de desarrollo, el curso admite *Visual Studio Code* (VSCode), que es un editor gratuito multi-plataforma con soporte de depuración. Se pueden utilizar otros editores pero sólo se proporciona soporte para VSCode.

Si desea instalar VSCode en su ordenador personal, puede descargarlo en <https://code.visualstudio.com/download> (a fecha de junio de 2025, la versión más actual 1.101.2 es compatible con este tutorial). Descargue y ejecute el fichero

para instalar correctamente VSCode en Windows. Tras instalar VSCode, deberá instalar el plugin *Python* para poder ejecutar código Python en VSCode. Este plugin puede instalarse directamente desde la interfaz de VSCode. No es necesario instalar el plugin *CodeRunner*. La instalación de plugins/extensiones se realiza a través del *extension marketplace*, con tutorial disponible en <https://code.visualstudio.com/docs/editor/extension-marketplace>

Puede consultar todos los pasos de instalación en el vídeo disponible en <https://recordit.co/cVo9YIBBOq>. Aunque este vídeo corresponde con la versión 1.35.1, es equivalente a la versión 1.101.2 disponible en la web.

1.3 Entorno de trabajo

Como la instalación y administración de librerías puede ser complicada, Python admite el uso de *entornos virtuales* (<https://docs.python.org/3.12/tutorial/venv.html>) que recopilan y aíslan un conjunto de bibliotecas paquetes en números de versión específicos. Python con todas las dependencias necesarias para este curso ya está instalado en los ordenadores del laboratorio en el entorno *practicatsv_env*.

1.3.1 Habilitando el entorno en el laboratorio

Para habilitar el entorno virtual en los ordenadores del laboratorio, abra un terminal de Windows y ejecute el siguiente comando:

```
$ source C:\Users\eps\practicatsv\practicatsv_env\Scripts\activate
```

Y posteriormente debe observar que el terminal ahora nos indica que está activo el entorno virtual:

```
$ (practicatsv_env)>
```

Además, puede desactivar el entorno virtual utilizando el siguiente comando:

```
$ deactivate
```

1.3.2 Creando un entorno en su ordenador personal

Adicionalmente puede recrear este entorno en su ordenador personal en el directorio *C:\Users\usuario* donde *usuario* es el usuario que utiliza en su ordenador. Verifique que tiene permisos de escritura en la ruta *C:\Users\usuario*.

Posteriormente descargue el fichero *requirements.txt* de la página Moodle del curso y ejecute las siguientes instrucciones:

```
$ python -m pip install --user virtualenv
$ mkdir C:\Users\usuario\practicatsv
$ python -m venv C:\Users\usuario\practicatsv\practicatsv_env
$ C:\Users\usuario\practicatsv\practicatsv_env\Scripts\activate
$ (practicatsv_env) python -m pip install --upgrade pip
$ (practicatsv_env) pip install -r requirements.txt
```

Pip es un administrador de paquetes que nos permite instalar fácilmente módulos de Python. En este escenario, el comando anterior lee una lista de módulos del fichero *requirements.txt* y los instala todos en el entorno virtual que se encuentra activo. Si aún no tiene *pip* instalado en su computadora, puede seguir el instrucciones en el siguiente enlace <https://pip.pypa.io/en/stable/installing/>.

NOTA: En otros sistemas operativos, es probable que no se ejecute correctamente la siguiente instrucción:

```
$ C:\Users\usuario\practicatsv\practicatsv_env\Scripts\activate
```

En ese caso, utilice el comando "source" de la siguiente manera:

```
$ source C:\Users\usuario\practicatsv\practicatsv_env\Scripts\activate
```

1.3.3 Habilitando el entorno en Visual Studio Code

Para utilizar el entorno virtual *practicatsv_env* en VSCode, debe cambiar el interprete de Python que se utiliza al ejecutar código. Para cambiar el intérprete, arranque VSCode y abra la directorio raíz donde está el entorno virtual. En nuestro caso, el entorno *practicatsv_env* se encuentra en el directorio `C:\Users\eps\Practicatsv`. Tras seleccionar el directorio, debe observar en la esquina inferior izquierda que el intérprete Python cambia automáticamente al entorno de trabajo que acabamos de seleccionar. La Figura 2 muestra un ejemplo de una habilitación exitosa del entorno de trabajo en VSCode.

Si queremos manejar varios entornos virtuales que existen en un mismo directorio. Pulse la combinación de teclas **Ctrl + Shift + P** y escriba **Python: Select Interpreter** en el cuadro de texto que aparece seleccionado. Posteriormente, debe observar un listado de los intérpretes Python disponibles en el sistema. Puede consultar todos los detalles del proceso en el enlace <https://code.visualstudio.com/docs/python/environments>

Si desea utilizar el entorno en su ordenador personal, primeramente deberá crear el entorno siguiendo los pasos descritos al final de la sección 1.3.1 anterior y posteriormente proceder como se ha indicado en esta sección.

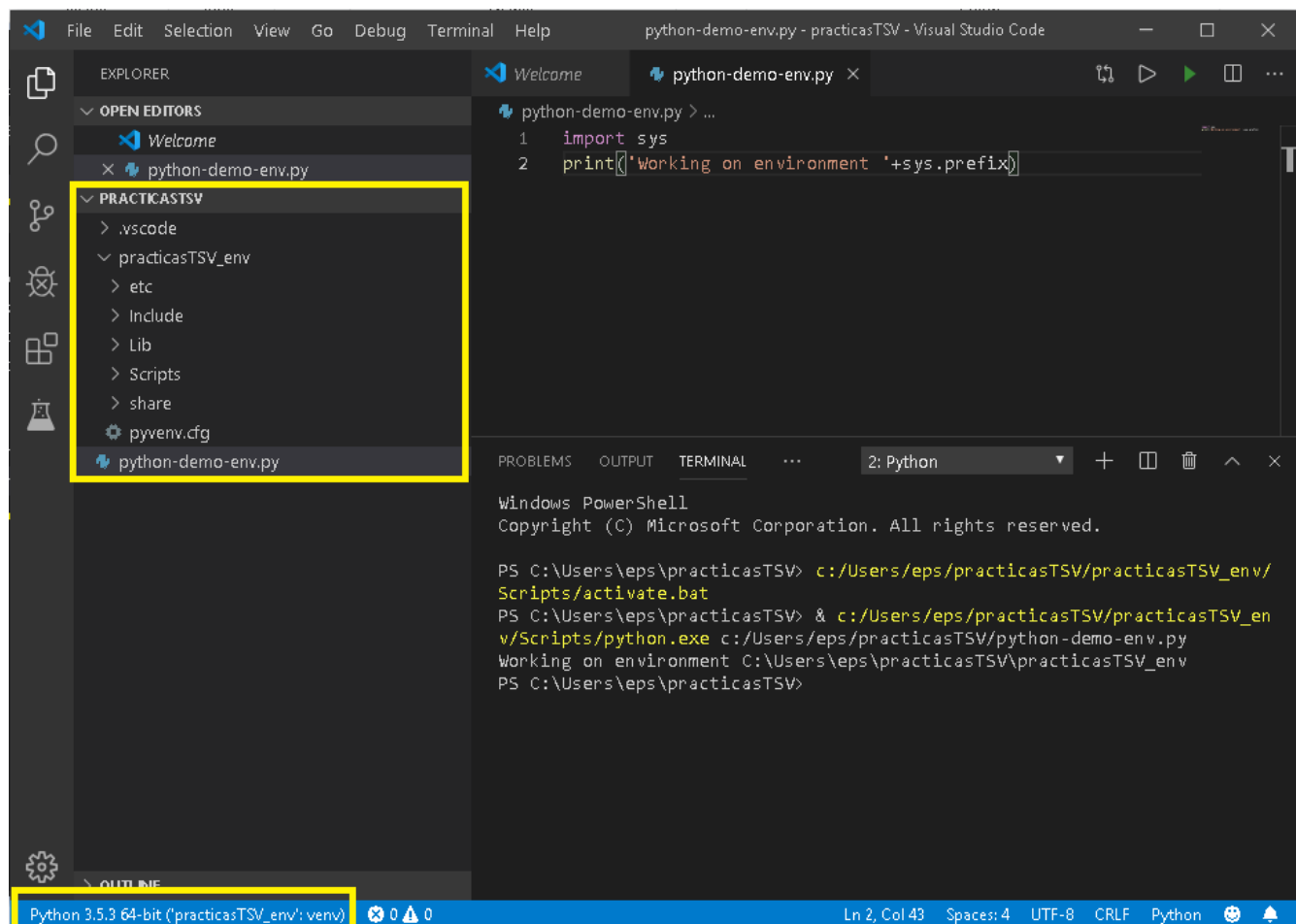


Fig. 2: VSCode mostrando una habilitación exitosa del entorno de trabajo *practicatsv_env*. Verifique que la información en los recuadros amarillos coincide con aquello que obtiene en su pantalla. En versiones posteriores de Visual Studio Code, el recuadro inferior izquierda puede aparecer en la derecha.

2 Ejecutando código

En esta sección se indica como puede ejecutar código en Python.

2.1 Terminal

El primer método que exploramos es la terminal (Windows o Linux). Como regla general, este documento utiliza el símbolo `$` para indicar comandos que se deben introducir en la terminal.

Debido a que en este curso se utiliza Windows, solamente proporcionaremos instrucciones para este sistema operativo. Abra la consola o terminal de comandos pulsando la combinación de teclas **Tecla Windows + R** y en la pantalla que aparece escriba `cmd` y presione **Enter**.

Para saber que versión de Python está usando, escriba `python -V` o `python`, en la consola de comandos. Debe obtener algo similar a la Figura 3 donde se puede observar que la versión es 3.12.0.

2.1.1 Modo interactivo

Tras abrir la consola y ejecutar el comando `python`, entramos en un modo interactivo de ejecución en Python en el cual podemos escribir y ejecutar líneas de código de manera interactiva. Podemos identificar que nos encontramos en el modo interactivo cuando observemos en la consola el símbolo `>>>`:

```
$ python
$ >>>
```

La Figura 3 muestra la línea de código `print('Hola')`.

2.1.2 Ejecución de ficheros

Si queremos ejecutar un fichero python desde la consola de comandos, entonces teclearemos `python myfichero.py`. Es decir, ejecutaremos Python sin entrar en el modo interactivo descrito anteriormente.

```
$ python myfichero.py
```

Tenga en cuenta que tanto `python` como `myfichero.py` deben ser accesibles desde la consola de comandos (por ejemplo, utilizando la variable de entorno `PATH`). Si esto no es posible, entonces se recomienda utilizar la ruta completa:

```
$ c:/Users/eps/practicatsv/practicatsv_env/Scripts/python.exe c:/Users/eps/practicatsv/myfichero.py
```

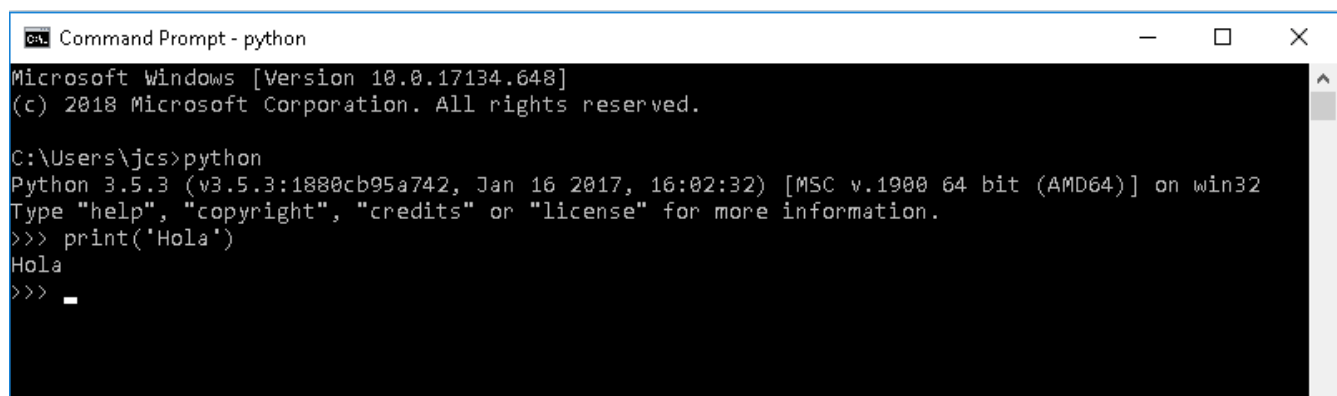


Fig. 3: Consola de Python en Windows

2.2 Visual Studio Code

El segundo método para ejecutar código que exploramos es mediante la herramienta *VSCode*.

2.2.1 Ejecución

Primeramente abriremos *VSCode* y después abrimos el entorno virtual de prácticas *practicatsv_env* siguiendo los pasos indicados en la sección 1.3.3.

Posteriormente, crearemos un nuevo fichero mediante con la combinación de teclas **Ctrl + N** y lo guardaremos con el nombre `python-ejemplo.py` pulsando la combinación de teclas **Ctrl + S** (ó **Ctrl + G** si el idioma de *VSCode* es español). Cuando guarde el fichero, se sugiere hacerlo en la ruta donde se encuentra nuestro entorno de trabajo (es decir, `C:\Users\eps\practicatsv`). La Figura 4 muestra el resultado del proceso descrito para crear un nuevo fichero.

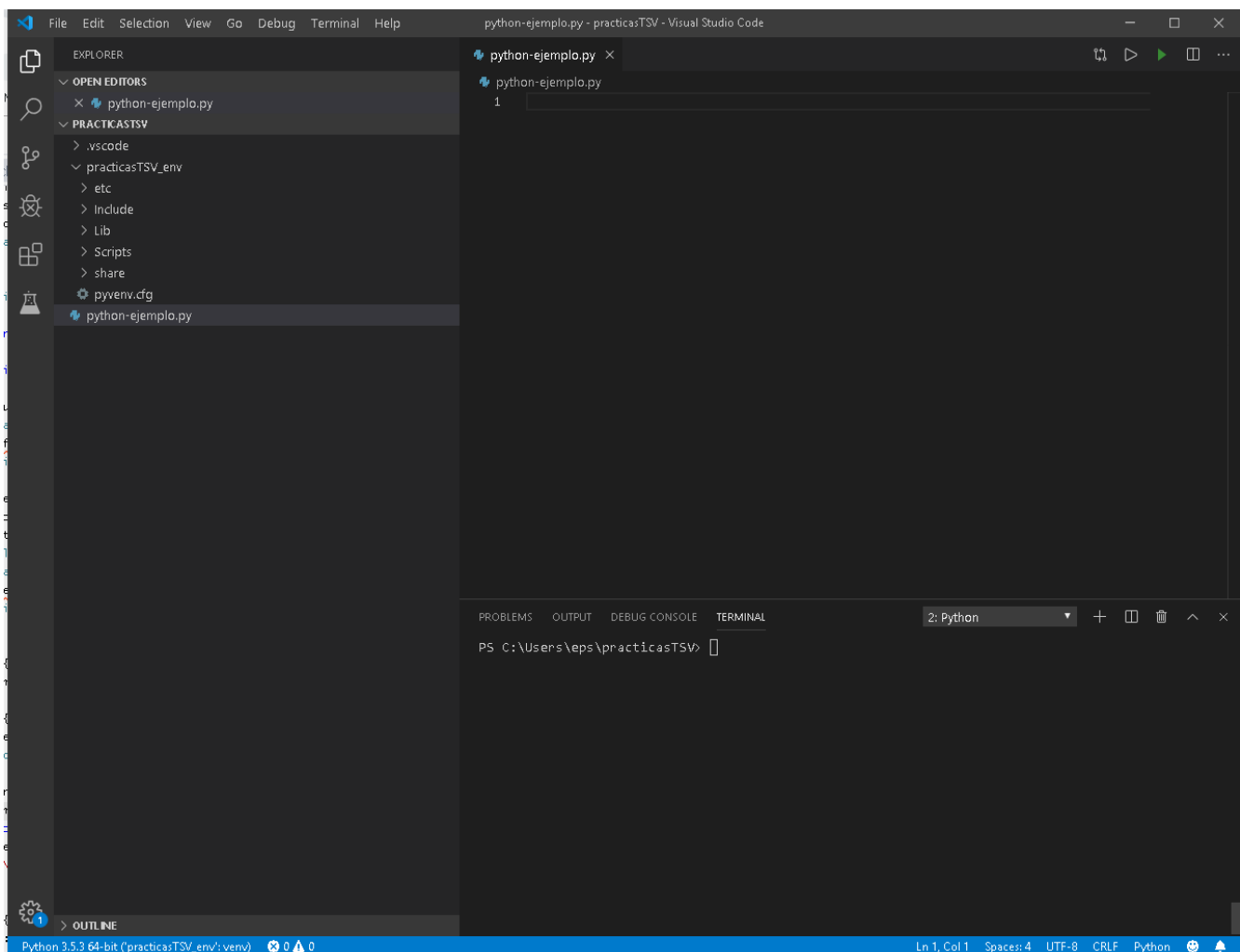


Fig. 4: VSCode mostrando la creación de un fichero en el entorno de trabajo *practicatsv_env*.

En este fichero que acaba de crear, escriba la instrucción `print('Hola Mundo')` y guárdelo pulsando **Ctrl + S** ó **Ctrl + G**. Después ejecute el fichero en el terminal, para ello haga clic con el botón derecho ratón sobre el fichero abierto y seleccionando la opción **Run Python File in Terminal**. Si observa el terminal (parte inferior de la interfaz), puede verificar que se ha ejecutado el código mostrando el texto `Hola Mundo`.

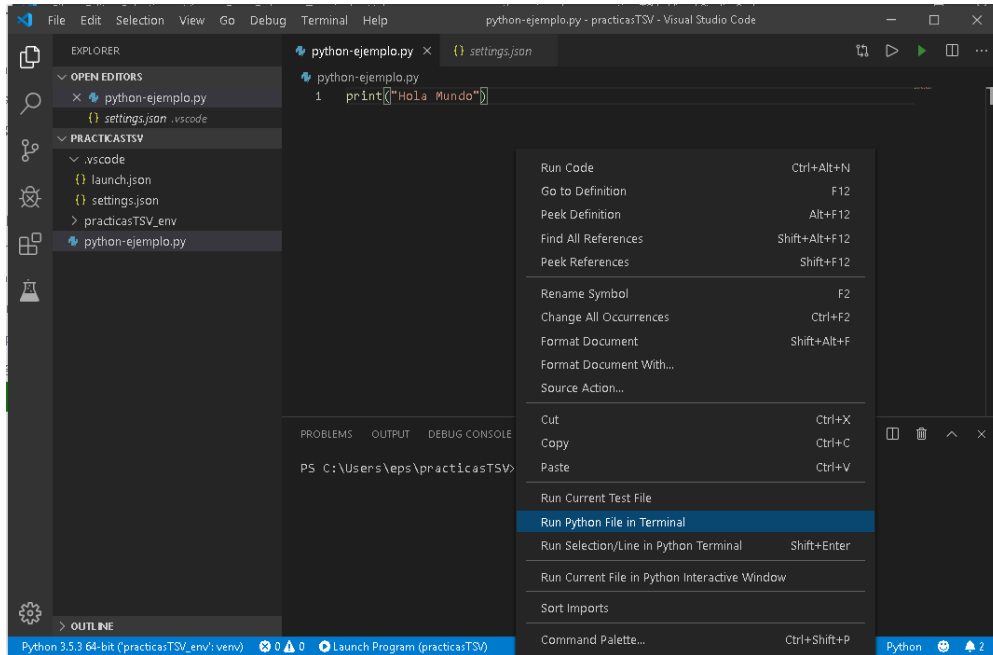


Fig. 5: Menu contextual en *VSCode* (botón derecho del ratón) para ejecutar un fichero de código en el terminal utilizando el entorno de trabajo *practicatsTV_env*.

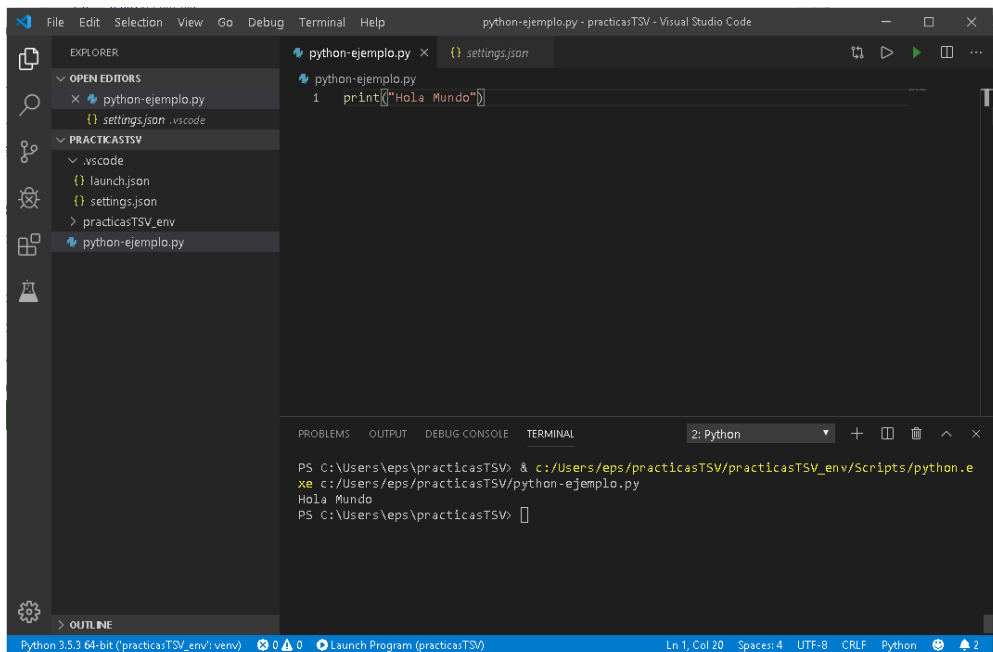


Fig. 6: Resultado de la ejecución del fichero de código en el terminal con la opción **Run Python File in Terminal** utilizando el entorno de trabajo *practicatsTV_env*.

2.2.2 Depuración

La depuración con *VSCode* permite parar la ejecución del código en una línea concreta para inspeccionar observar el valor de las variables en ese momento.

Para seleccionar en que líneas queremos que el código se pare (llamados *breakpoints*), posicione el ratón a la izquierda de la línea de código deseada y verá que aparece un círculo rojo. Haga click con el botón derecho del ratón y observe como aparecen diversas opciones. Seleccione *Add breakpoint* y observe como el círculo rojo se queda fijo indicando que hemos establecido un *breakpoint*. La Figura 7 muestra el resultado tras crear un *breakpoint*.

Para ejecutar el depurador simplemente pulse la tecla **F5** y en el menú que aparece seleccione *Python Debug the currently active Python file*. Tras ello, automáticamente comenzará el programa y se parará en la línea seleccionada. La Figura 8 muestra el resultado tras arrancar el depurador.

Puede consultar más información sobre depuración en <https://code.visualstudio.com/docs/editor/debugging/>.

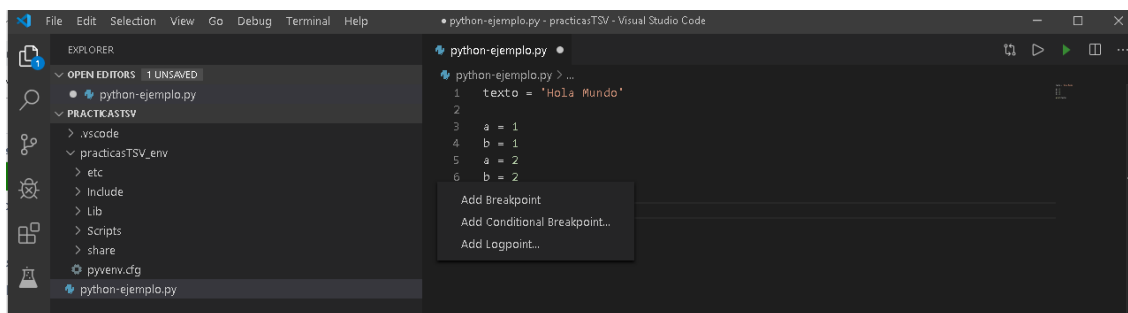


Fig. 7: VSCode mostrando la creación de *breakpoint* en un fichero de código.

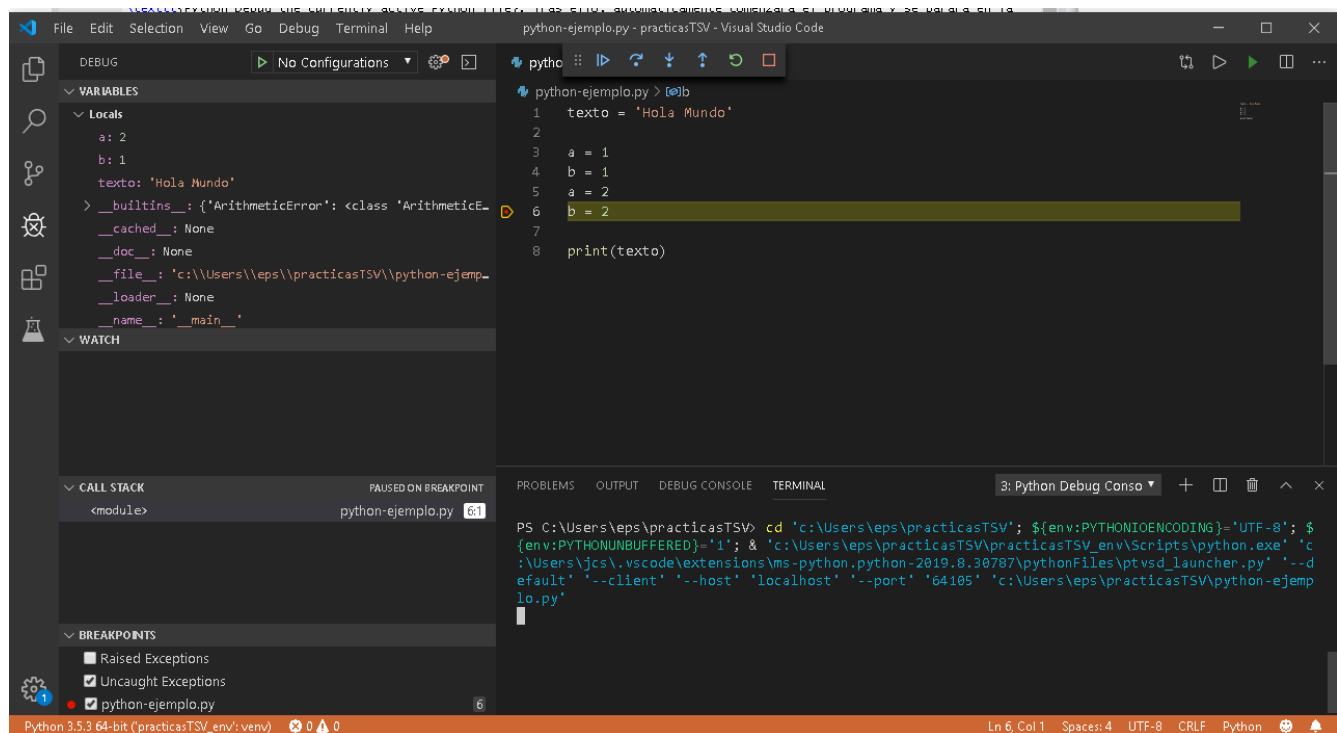


Fig. 8: VSCode mostrando la ejecución del depurador para el *breakpoint* en la Figura 7. En el apartado de la izquierda podemos observar las variables y sus valores (sección *VARIABLES*). Además, podemos añadir variables a monitorizar en la sección *WATCH*.

Siga las siguientes instrucciones si no consigue que el código se pare en los breakpoints definidos o el depurador no arranca y/o no observa ninguna salida en consola.

1. Primeramente diríjase a la página de plugins y comprobar que el plugin de Python es una versión posterior a 2022.8.1 (ver Figura 9).
2. Haga click en el símbolo localizado a la derecha del botón uninstall. Después seleccione "Install Another Version..." (ver Figura 10).
3. Haga click en la versión 2022.8.1 para instalarla (ver Figura 11).
4. Posteriormente verá como instala la versión seleccionada.
5. Al completarse, verá como se requiere reiniciar VSCode (ver Figura 12).
6. Tras reiniciar VSCode, vuelva a la página de plugins para verificar que la versión instalada sea la 2022.8.1 (ver Figura 13).
7. Vuelva a probar los pasos para depurar siguiendo los siguientes tutoriales <https://code.visualstudio.com/docs/editor/debugging> y <https://code.visualstudio.com/python/debugging/>.

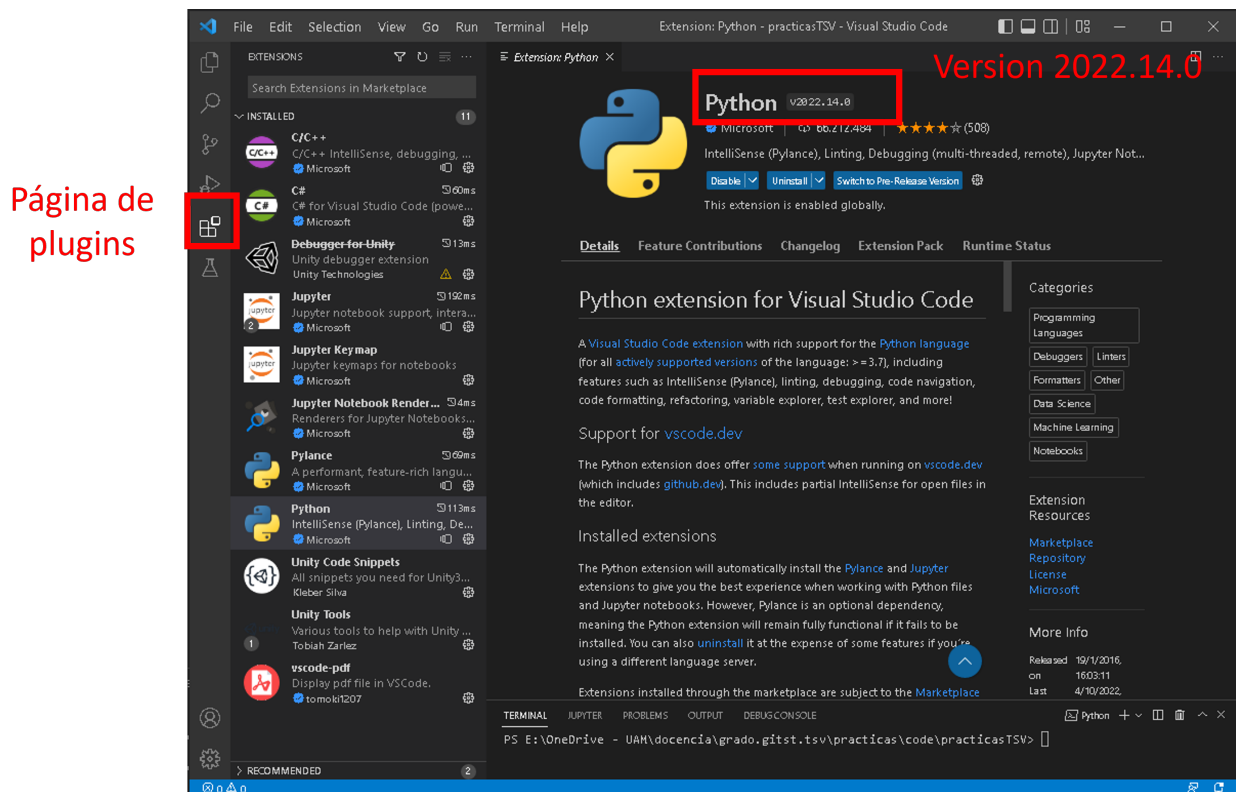


Fig. 9: VSCode mostrando la versión del plugin de Python.

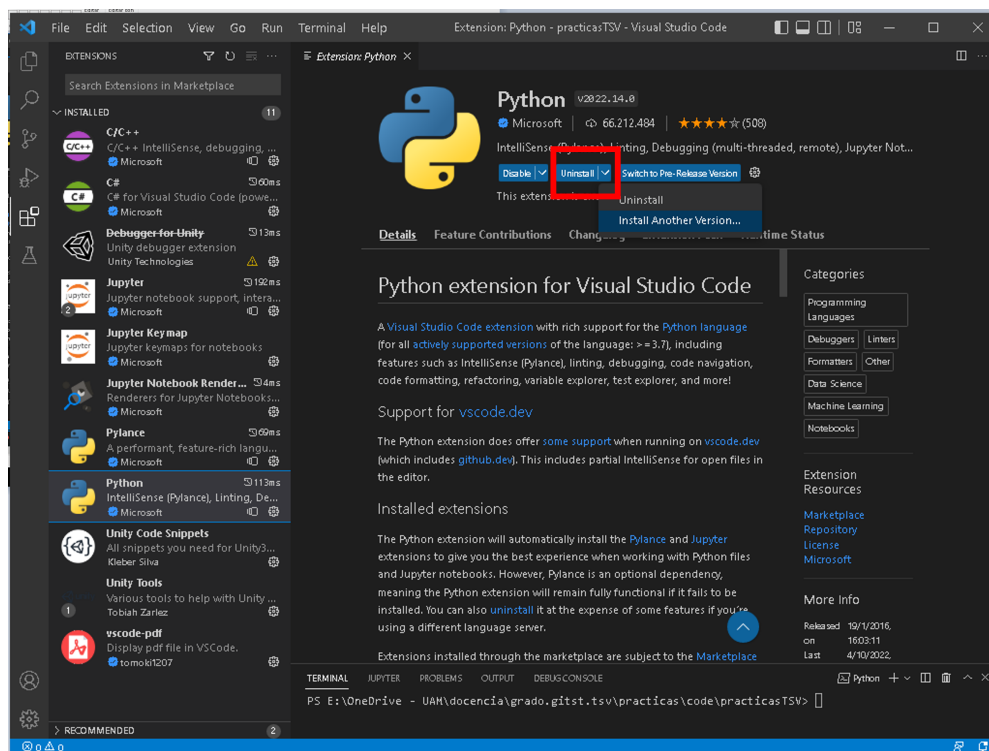


Fig. 10: VSCode botón para cambiar la versión del plugin de Python.

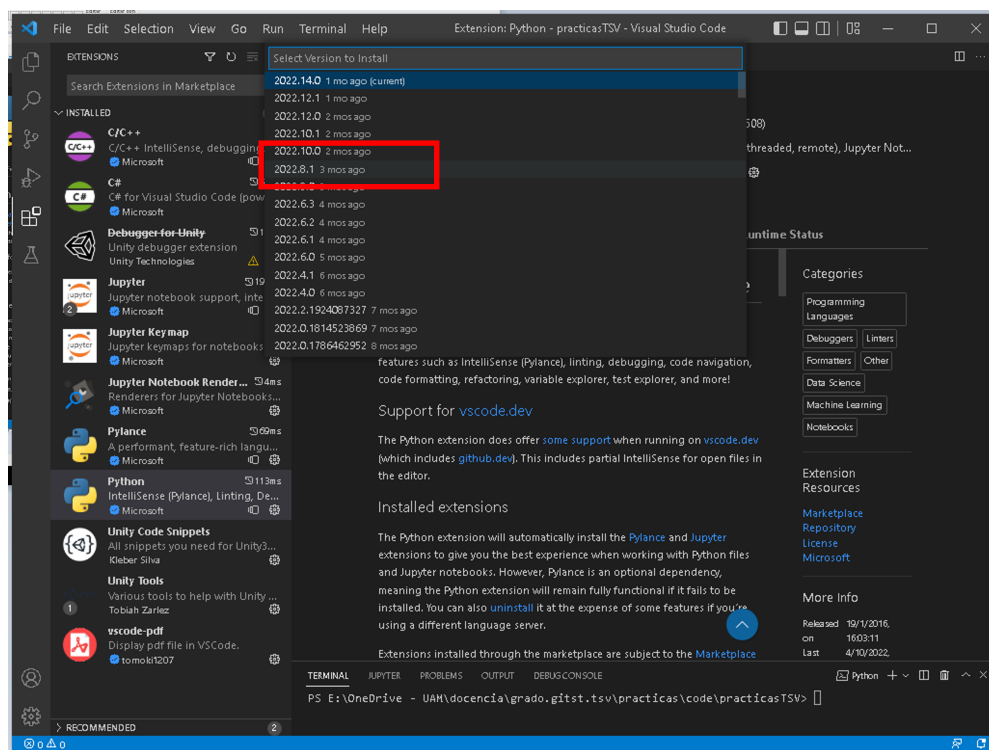


Fig. 11: VSCode versión del plugin de Python compatible con depurar en Python 3.6.8.

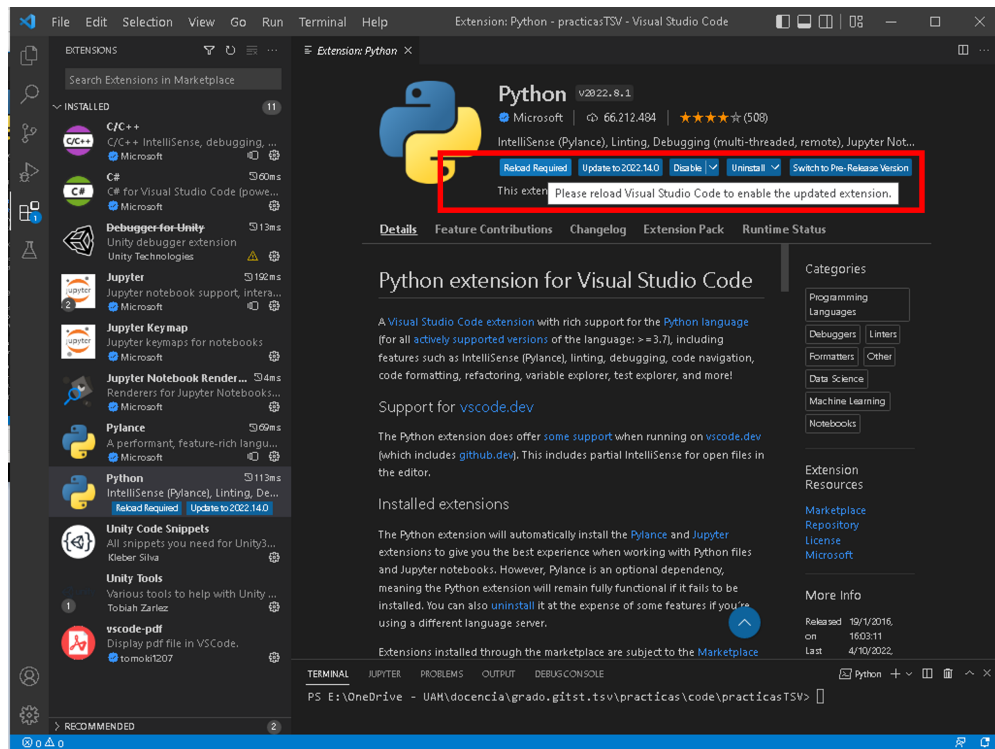


Fig. 12: VSCode reinicio requerido tras cambiar el plugin de Python.

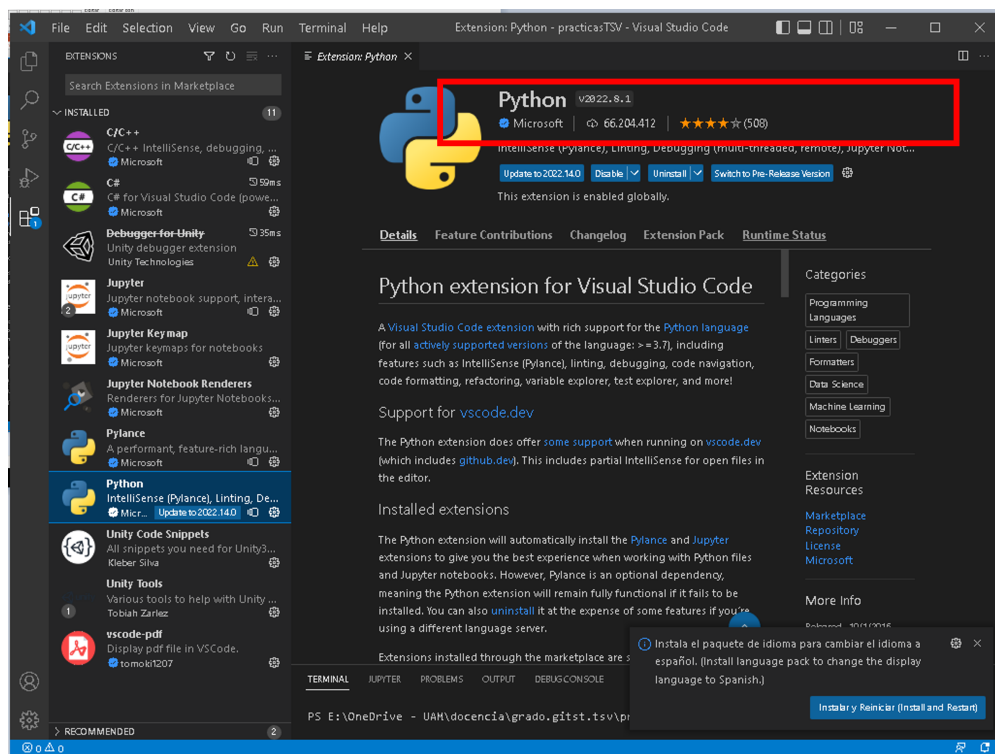


Fig. 13: VSCode reinicio requerido tras cambiar el plugin de Python.

Adicionalmente, si tras seguir este tutorial no consigue que el depurador de VSCode se pare en una línea concreta, un error común es la localización de los ficheros de código Python. Estos ficheros *.py no deben estar en la misma carpeta *PracticasTSV_env* y por ello no funciona el depurador.

Aunque estos ficheros *.py pueden almacenarse en varias localizaciones, se sugiere la siguiente estructura de directorios (ver Figura 14):

```
<mi_directorio>/PracticasTSV
<mi_directorio>/PracticasTSV/PracticasTSV_env --> librerías y utilidades de Python instaladas
<mi_directorio>/PracticasTSV/P0 --> código práctica 0
<mi_directorio>/PracticasTSV/P1 --> código práctica 1
<mi_directorio>/PracticasTSV/P2 --> código práctica 2
...
```

Si se emplea esta estructura, al abrir la carpeta *mi_directorio/PracticasTSV* con VSCode verá como automáticamente se carga la versión de Python en *mi_directorio/PracticasTSV/PracticasTSV_env* sin ninguna configuración adicional.

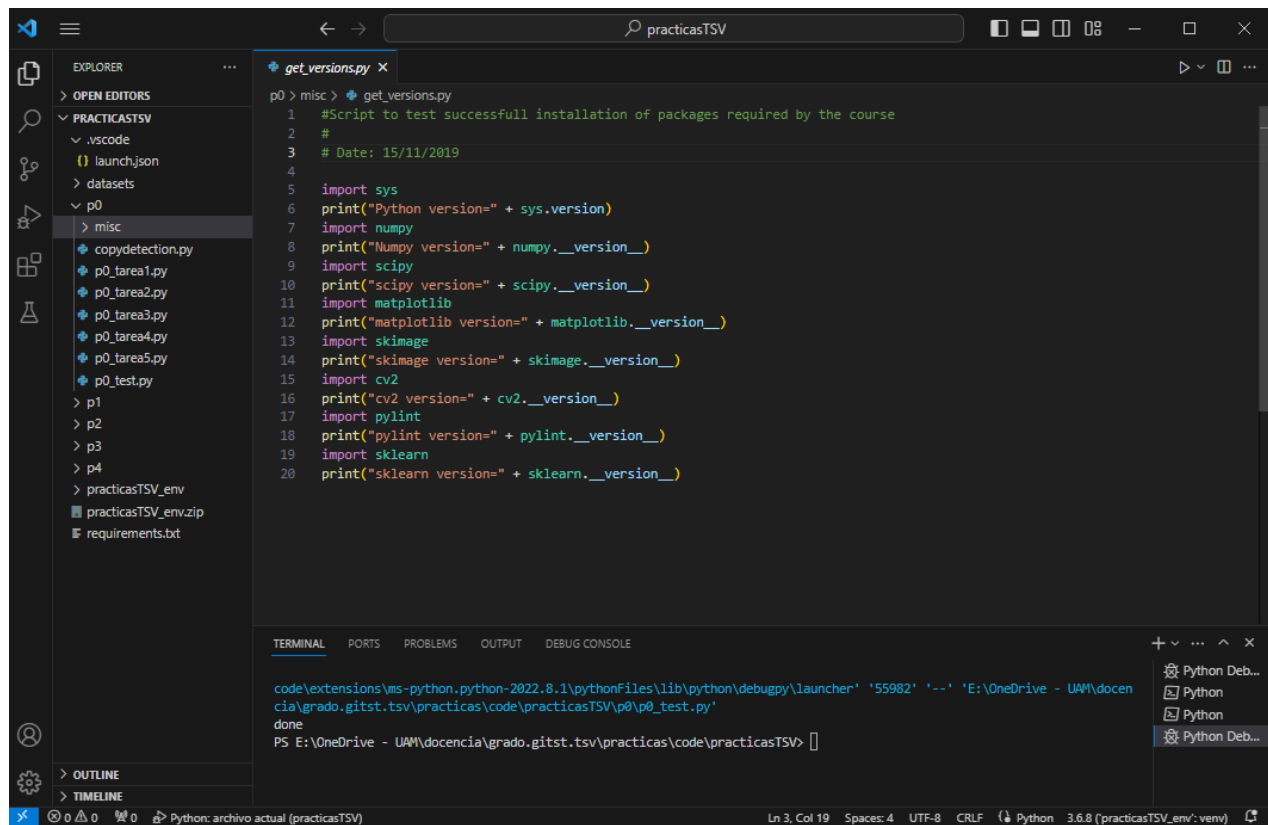


Fig. 14: Sugerencia de estructura de directorios para VSCode, que automáticamente carga la versión correcta de Python..

3 Aspectos básicos

La documentación general de Python 3.6.8 está disponible en <https://docs.python.org/3.6/>. En general la calidad de la documentación de Python es excelente y se recomienda su uso de manera frecuente.

Si es la primera vez que utiliza Python, se recomienda realizar el tutorial oficial de Python disponible en la web <https://docs.python.org/3.6/tutorial/>. En esta sección se han seleccionado conceptos básicos relacionados con el curso. Familiarícese con ellos y pruebe los ejemplos de código con *VSCode*. Si tiene alguna pregunta, no dude en consultar con su profesor.

En cada apartado se le proporcionan pequeños ejemplos de código, se recomienda utilizar *VSCode* para verificar y comprender cada ejemplo. Por ejemplo, copie cada extracto de código en el fichero `python-ejemplo.py` creado tras completar los pasos indicados en la sección 1.3.3.

3.1 Sangría

Tenga en cuenta las reglas de espacios en blanco (sangrado de párrafos o sangría) para un correcto funcionamiento de su código. Python no requiere corchetes para separar métodos y clases. El flujo de control del código se declara con sangría. El siguiente ejemplo de código no funcionará debido al error de sangría en el bloque de código de la instrucción `if`.

```
x = 1
y = 2
if x < y:
x=3 # Error de sangrado!
```

pero este código corregido si que ejecutará

```
x = 1
y = 2
if x < y:
    x=3 # Error eliminado con espacios en blanco!
```

Cada nivel de sangría generalmente se indica mediante un tabulador o con 4 espacios en blanco. Puede elegir cualquier opción pero sea coherente en su fichero.

3.2 Comentarios

En Python los comentarios se pueden de dos formas:

- Escribiendo el símbolo almohadilla delante de la línea de texto donde está el comentario.
- Escribiendo triple comilla doble (`"""`) al principio y al final del comentario (que puede ocupar más de una línea).

```
""" Comentario que puede ocupar
    varias líneas """
x = 1 # Comentario en una línea
y = 2
```

3.3 Mostrar datos en el terminal

Para mostrar texto en el terminal, se utiliza el comando `print` que permite mostrar tanto cadenas de caracteres como el contenido de variables. A continuación, se muestran varios ejemplos de `print`:

```
print("Texto de Ejemplo 1") # muestra "Texto de Ejemplo 1"
x = 3
print("Texto de Ejemplo " + str(x)) # muestra "Texto de Ejemplo 3"

print(type(x)) # muestra "<class 'int'>"
print(x)        # muestra "3"
print(x + 1)     # Suma; muestra "4"
print(x - 1)     # Resta; muestra "2"
print(x * 2)     # Multiplicacion; muestra "6"
print(x ** 2)    # Exponenciacion; muestra "9"
```

3.4 Variables

3.4.1 Tipos básicos

Números (Numbers): en Python declaramos números enteros y reales de manera sencilla:

```
x = 3 # numero entero
x = int(3) # numero entero
x = 3.0 # numero real
x = float(3.0) # numero real - float
```

Al contrario que C++/Matlab, Python no soporta las operaciones de incremento (`x++`) or decremento (`x--`).

Booleanos (Booleans): Python implementa todos los operadores habituales para la lógica booleana, pero usa palabras en inglés en lugar de símbolos (&&, —, etc.):

```
t = True
f = False
print(type(t)) # muestra "<class 'bool'>"
print(t and f) # Logical AND; muestra "False"
print(t or f)  # Logical OR; muestra "True"
print(not t)   # Logical NOT; muestra "False"
print(t != f)  # Logical XOR; muestra "True"
```

Texto (String): Python proporciona un gran soporte para trabajar con texto (cadenas de caracteres)

```
hello = 'hello' # texto puede declararse con comillas simples
world = "world" # o con comillas dobles (es equivalente).
print(hello)    # muestra "hello"
print(len(hello)) # muestra la longitud del texto; prints "5"
hw = hello + ' ' + world # concatenar texto
print(hw)        # muestra "hello world"
hw12 = '%s %s %d' % (hello, world, 12) # formateo al estilo sprintf
print(hw12)      # muestra "hello world 12"
```

y además incluye varias métodos de utilidad para el manejo de cadenas de caracteres:

```
s = "hello"
print(s.capitalize()) # Texto capitalizado; muestra "Hello"
print(s.upper())      # Text en mayusculas; muestra "HELLO"
print(s.rjust(7))     # Justificacion a la derecha, rellena con espacios en blanco hasta tener 7
                      # caracteres; muestra " hello"
print(s.center(7))    # Muestra el texto centrado, rellena con espacios hasta tener 7; muestra "
                      # hello "
print(s.replace('l', '(ell)')) # Sustituir todas las 'l' con '(ell)', muestra "he(ell)(ell)o"
```

Puede encontrar una listado con los métodos en <https://docs.python.org/3.6/library/stdtypes.html>

3.4.2 Contenedores (containers)

Por defecto, Python incluye cuatro tipos de contenedores: listas, diccionarios, conjuntos y tuplas. En esta sección analizamos solamente los tres primeros, al ser los tipos de contenedores más utilizados.

Listas (lists): En Python, una lista es equivalente a un array que puede cambiar de dimensión y contener datos de diferentes tipos. Si la lista es unidimensional (e.g., `xs[...]`), entonces su uso se asimilará a un vector. Mientras que si la lista es multidimensional (e.g., `xs[...][...]` o `xs[...][...][...]`), entonces su uso es similar a un array.

```
xs = [3, 1, 2]      # crear una lista con enteros
print(xs, xs[2])   # muestra "[3, 1, 2] 2"
print(xs[-1])      # indices negativos cuentan desde el final de la lista; muestra "2"
xs[2] = 'foo'      # Lista puede contener elementos de distintos tipos
print(xs)          # muestra "[3, 1, 'foo']"
xs.append('bar')    # añadir un nuevo elemento al final de lista
print(xs)          # muestra "[3, 1, 'foo', 'bar']"
x = xs.pop()        # eliminar y devolver el ultimo elemento de la lista
x = xs.pop(1)       # eliminar y devolver el segundo elemento de la lista
print(x, xs)        # muestra "bar [3, 1, 'foo']"
xs.insert(2, 'n')    # añadir elemento en posicion 2 y mover el resto de elementos
print(xs)          # muestra "[3, 1, 'n', 'foo']"
xs.remove('n')       # elimina la primera vez que aparece el elemento 'n'
print(xs)          # muestra "[3, 1, 'foo']"
print('Num elementos = ' + str(len(xs))) # muestra "Num elementos = 3"
```

El acceso a varios elementos de una lista puede realizarse de las siguientes maneras:

```
nums = list(range(5)) # range es una funcion que crea una lista de enteros desde 0 hasta 4
print(nums)           # muestra "[0, 1, 2, 3, 4]"
print(nums[2:4])      # obtener los elementos de la posicion 2 al 4 (sin incluir); muestra "[2, 3]"
print(nums[2:])        # obtener los elementos de la posicion 2 al final; muestra "[2, 3, 4]"
print(nums[:2])        # obtener los elementos del inicio hasta la posicion 2 (excluida); muestra "[0, 1]"
print(nums[:])         # obtener todos los elementos; muestra "[0, 1, 2, 3, 4]"
print(nums[::-1])      # los indices pueden ser negativos; muestra "[4, 3, 2, 1, 0]"
nums[2:4] = [8, 9]     # sobrescribir los elementos de la posicion 2 y 3
print(nums)            # muestra "[0, 1, 8, 9, 4]"
```

Los bucles sobre listas se pueden realizar de dos maneras:

```
# Primera opcion
animals = ['cat', 'dog', 'monkey']
for animal in animals:
    print(animal)
# muestra "cat", "dog", "monkey", cada uno en una linea distinta

# Segunda opcion - idx indica la posicion en la lista
animals = ['cat', 'dog', 'monkey']
for idx, animal in enumerate(animals):
    print('#%d: %s' % (idx + 1, animal))
# Prints "#1: cat", "#2: dog", "#3: monkey", cada uno en una linea distinta
```

La documentación completa sobre la funcionalidad de listas en Python puede consultarse en el siguiente enlace <https://docs.python.org/3.6/tutorial/datastructures.html>

Diccionarios (Dictionaries)¹: contenedores donde los elementos no tienen orden y se almacenan parejas (clave, valor) conocidas como "entradas". A continuación se muestran varios ejemplos de uso:

```
d = {'cat': 'cute', 'dog': 'furry'} # creacion de diccionario
print(d['cat'])                    # obtener una entrada del diccionario; muestra "cute"
print('cat' in d)                  # verificar si existe la clave en el diccionario; muestra "True"
d['fish'] = 'wet'                  # incluir una nueva entrada en el diccionario
print(d['fish'])                   # muestra "wet"
# print(d['monkey'])               # KeyError: 'monkey' no es una clave de d
d.update({'cat': 'furry'})         # insertar una determinada clave o actualiza su valor si ya existiera.
print(d.get('monkey', 'N/A'))     # obtener un elemento con valor por defecto; muestra "N/A"
print(d.get('fish', 'N/A'))       # obtener un elemento con valor por defecto; muestra "wet"
del d['fish']                      # eliminar un elemento del diccionario
print(d.get('fish', 'N/A'))       # "fish" ya no es una clave; muestra "N/A"
```

Los bucles sobre diccionarios se realizan de manera similar a las listas:

```
# Opcion 1 - acceso al valor
d = {'person': 2, 'cat': 4, 'spider': 8}
for animal in d:
    legs = d[animal]
    print('A %s has %d legs' % (animal, legs))
# muestra "A person has 2 legs", "A cat has 4 legs", "A spider has 8 legs"

# Opcion 2 - acceso a la clave y valor
d = {'person': 2, 'cat': 4, 'spider': 8}
for animal, legs in d.items():
    print('A %s has %d legs' % (animal, legs))
# Prints "A person has 2 legs", "A cat has 4 legs", "A spider has 8 legs"
```

Conjuntos (sets): aunque menos frecuentes, Python también dispone de conjuntos que definen una colección de elementos sin orden. A continuación, se muestran ejemplos ilustrativos del manejo de conjuntos:

```
animals = {'cat', 'dog'}
print('cat' in animals)          # Verifica si un elemento esta en el conjunto; muestra "True"
print('fish' in animals)         # muestra "False"
animals.add('fish')               # Anadir un elemento al conjunto
print('fish' in animals)         # muestra "True"
print(len(animals))               # Numero de elementos en el conjunto; muestra "3"
animals.add('cat')                # Anadir un elemento al conjunto que ya existe --> no se hace nada
print(len(animals))               # muestra "3"
animals.remove('cat')             # Eliminar un elemento del conjunto
print(len(animals))               # muestra "2"
```

Las operaciones más habituales entre conjuntos que se pueden realizar en Python son:

- $A \cup B$: Unión entre el conjunto A y B (suma de elementos).
- $A \cap B$: Intersección entre el conjunto A y B.
- $A - B$: Diferencia entre el conjunto A y B.

Los bucles sobre conjuntos se realizan de manera similar a las listas. No obstante, no se puede hacer ninguna asunción acerca del orden en el cual se extraen los elementos:

```
animals = {'cat', 'dog', 'fish'}
for idx, animal in enumerate(animals):
    print('#%d: %s' % (idx + 1, animal))
# muestra "#1: fish", "#2: dog", "#3: cat"
```

¹A partir de la versión 3.7 de Python, los diccionarios están ordenados. En Python 3.6 y anteriores, los diccionarios no están ordenados (consultar https://www.w3schools.com/PYTHON/python_dictionaries.asp). No obstante, para la asignatura no es un factor relevante la ordenación (o no) de los diccionarios.

3.5 Condiciones

En Python, se utiliza el comando `if` para indicar las condicionales que pueden bifurcar nuestro código. Adicionalmente, existen los opcionales `elif` y `else` que se utilizan de manera similar a otros lenguajes de programación. A continuación, se muestran ejemplos ilustrativos:

```
nota_alumno=5.5
if (nota_estudiante==10.0):
    print("El estudiante ha obtenido matricula de honor")
elif (nota_alumno>=5.0):
    print("El estudiante ha aprobado ")
else:
    print("El estudiante ha suspendido")
```

Se puede observar que las líneas dentro de cada condición tienen la misma sangría (4 espacios blancos a la izquierda). Las condiciones que se suelen usar con más frecuencia son:

- $a == b$ Indica si a es igual a b
- $a < b$ Indica si a es menor a b
- $a > b$ Indica si a es mayor a b
- $not(a > b)$ Niega la condición ($a > b$)
- $a < b$ and $c > b$ 'and' junta dos condiciones ($a > b$) y ($c > b$). Deben cumplirse ambas
- $a < b$ or $c > b$ 'or' junta dos condiciones ($a > b$) y ($c > b$). Debe cumplirse una de ellas o ambas

3.6 Bucles

En Python, los bucles se implementan con los comandos `while` y `for`.

El comando `while` define un bucle en base a una condición. A continuación, se muestra un ejemplo ilustrativo:

```
valor=1
while valor<5:
    print("Valor "+str(valor))
    valor=valor+1
# muestra "Valor 1", "Valor 2","Valor 3","Valor 4" en líneas distintas
```

El comando `for` permite definir un bucle que itera sobre elementos. A continuación, se muestra un ejemplo ilustrativo:

```
for valor in range(1,5):
    print("Valor "+str(valor))
# muestra "Valor 1", "Valor 2","Valor 3","Valor 4" en líneas distintas
```

La función `range` puede emplearse de maneras distintas, que se muestran a continuación:

- `range(stop)`
- `range(start, stop[, step])`

En la función `range` los argumentos deben ser enteros. Si se omite el argumento `step`, por defecto es 1. Si se omite el argumento `start`, por defecto es 0. Puede consultar más información en <https://docs.python.org/2/library/functions.html#range>

El comando `for` se puede utilizar sobre cualquier elemento iterable como son listas, diccionarios, conjuntos y tuplas. A continuación, se muestran ejemplos ilustrativos:

```
# Ejemplo de iteracion sobre una lista
coches = ('Ferrari', 'Tesla', 'BMW', 'Audi')
for coche in coches:
    print(coche)
# muestra 'Ferrari', 'Tesla', 'BMW' y 'Audi' cada uno en una linea

# Ejemplo de iteracion sobre una lista y asignar un numero de orden
coches = ('Ferrari', 'Tesla', 'BMW', 'Audi')
for i, coche in enumerate(coches):
    print(str(i) + " - " + coche)
# muestra '0 - Ferrari', '1 - Tesla', '2 - BMW' y '3 - Audi' cada uno en una linea
```

3.7 Funciones

Las funciones en Python se definen utilizando el comando `def`. A continuación, se muestra un ejemplo ilustrativo:

```
#definicion de la funcion signo
def sign(x):
    if x > 0:
        return 'positive'
    elif x < 0:
        return 'negative'
    else:
        return 'zero'

#codigo que ejecuta la funcion signo
for x in [-1, 0, 1]:
    print(sign(x))
# muestra "negative", "zero", "positive"
```

Observe que las funciones han de definirse siempre antes del código que las utiliza.

Adicionalmente, también se pueden definir funciones con argumentos opcionales de la siguiente manera:

```
def hello(name, loud=False):
    if loud:
        print('HELLO, %s!' % name.upper())
    else:
        print('Hello, %s!' % name)

hello('Bob')           # muestra "Hello, Bob"
hello('Fred', loud=True) # muestra "HELLO, FRED!"
hello('Fred', True)    # muestra "HELLO, FRED!"
```

La documentación completa sobre el uso de funciones en Python puede consultarse en el siguiente enlace

<https://docs.python.org/3.6/tutorial/controlflow.html>

3.8 Clases

La sintaxis para definir clases en Python es sencilla:

```
#definicion de la clase Greeter
class Greeter():

    # Constructor
    def __init__(self, name):
        self.name = name # Create an instance variable

    # Instance method
    def greet(self, loud=False):
        if loud:
            print('HELLO, %s!' % self.name.upper())
```

```

        else:
            print('Hello, %s' % self.name)

# codigo que crea una instancia de la clase y ejecuta un metodo
g = Greeter('Fred') # Crear un instancia de la clase Greeter
g.greet()           # llamada al metodo greet; muestra "Hello, Fred"
g.greet(loud=True)  # llamada al metodo greet; muestra "HELLO, FRED!"

```

La documentación sobre el uso de clases puede consultarse en <https://docs.python.org/3.6/tutorial/classes.html>

3.9 Uso de paquetes

Un paquete, módulo o librería son una serie de funciones que vienen empaquetados para ser usado por otros programas de manera sencilla. Python dispone de paquetes para realizar multitud de operaciones. Para listar de paquetes instalados y versiones, ejecuta los siguientes comandos:

```

$ pip list                #listado de todos los paquetes instalados
$ python -m pip list      #listado de todos los paquetes instalados
$ pip list | grep matplotlib #este comando solo funciona en Unix

```

Para utilizar un paquete debemos utilizar el comando `import` y posteriormente ya podemos acceder a toda la funcionalidad del paquete. A continuación, se muestra un ejemplo para librería de sistema `sys` y su método `sys.prefix`:

```

import sys
print('Environment ' + sys.prefix) # muestra el entorno donde se trabaja

import sys as s #cargamos la libreria con un nombre corto 's'
print('Environment ' + s.prefix) # muestra el entorno donde se trabaja

```

Para listar la versión de un paquete instalado, podemos utilizar el método `__version__` o `version` disponible en la mayoría de paquetes y ejecutar las siguientes instrucciones en Python.

```

import sys
import matplotlib

print('sys: ' + sys.version) # muestra "sys: 3.6.8"
print('matplotlib: ' + matplotlib.__version__) # muestra "matplotlib: 3.0.3"

```

Para instalar un nuevo paquete, podemos ejecutar las siguientes instrucciones:

```

$ python -m pip install matplotlib          # instalar paquete matplotlib
$ python -m pip install matplotlib==3.0.3  # instalar paquete matplotlib con version especifica
$ pip install -r requirements.txt           # instalar paquetes definidos en fichero

```

Cabe recordar que si queremos instalar los paquetes en nuestro entorno de trabajo *practicatsv_env*, primero debe estar activo y posteriormente puede instalar los paquetes que sean necesarios:

```

$ C:\Users\eps\practicatsv\practicatsv_env\Scripts\activate #activar entorno en Windows
$ (practicatsv_env)python -m pip install matplotlib #instalar paquete
$ (practicatsv_env)python -m pip install matplotlib==3.0.3 #instalar paquete especifico
$ (practicatsv_env)pip install -r requirements.txt #instalar paquetes de fichero

```

3.10 Ficheros

En Python, puede diseñar su código de manera que utilice múltiples ficheros. Para ello, observe el siguiente ejemplo:

Fichero: `hello.py`

```
def hello():  
    print 'hello'  
  
def bye():  
    print 'Bye'
```

Fichero: `main.py`

```
from hello import *           #importa todas las funciones del fichero hello  
from hello import hello, bye  #importa solo las funciones hello,bye del fichero hello  
  
hello()                       #muestra 'hello'  
bye()                         #muestra 'bye'
```

4 Paquetes

4.1 Numpy

Numpy es la principal biblioteca para el cálculo científico en Python. Proporciona funcionalidad para procesamiento de matrices multidimensionales. Existen varias maneras para importar la funcionalidad de Numpy en su fichero de código. A continuación se muestran varios ejemplos

```
#Opcion 1 -
import numpy
numpy.X                #acceso a la funcion/objeto 'X' de numpy

#Opcion 2 (aconsejado)
import numpy as np     #importar numpy con el nombre corto 'np'
np.X                   #acceso a la funcion/objeto 'X' de numpy

#Opcion 3 (desaconsejado)
from numpy import *    #importar todas las funciones/objetos
X                       #ya no necesitamos usar numpy.X o np.X para acceder
```

4.1.1 Arrays

Al contrario en que las listas de Python, un *Array* de Numpy es un conjunto de elementos del mismo tipo. Puede consultar la documentación oficial en <https://docs.scipy.org/doc/numpy/user/basics.creation.html#arrays-creation>

A continuación, se muestran varios ejemplos para **iniciar y acceder a los elementos de matrices**:

```
import numpy as np

a = np.array([1, 2, 3]) # crear un array 3x1
print(type(a))         # muestra "<class 'numpy.ndarray'>"
print(a.shape)         # muestra "(3,)"
print(a[0], a[1], a[2]) # muestra "1 2 3"
a[0] = 5               # modificar un elemento del array
print(a)               # muestra "[5, 2, 3]"

b = np.array([[1,2,3],[4,5,6]]) # crear un array 2x3
print(b.shape)                 # muestra "(2, 3)"
print(b[0, 0], b[0, 1], b[1, 0]) # muestra "1 2 4"
```

Al contrario que MATLAB, observe que el primer elemento de un array en Numpy se indica con la posición 0.

A continuación, se muestran varios ejemplos para **iniciar matrices**:

```
import numpy as np

a = np.zeros((2,2)) # crea un array de ceros
print(a)            # muestra "[[ 0.  0.]
                    #          [ 0.  0.]]"

b = np.ones((1,2)) # crea un array de unos
print(b)           # muestra "[[ 1.  1.]]"

c = np.full((2,2), 7) # crea un array de valores constantes
print(c)              # muestra "[[ 7.  7.]
                    #          [ 7.  7.]]"

d = np.eye(2) # crea un array con una matriz identidad 2x2
print(d)      # muestra "[[ 1.  0.]
                    #          [ 0.  1.]]"

e = np.random.random((2,2)) # crea un array con valores aleatorios
print(e)                    # puede mostrar "[[ 0.91940167  0.08143941]
                    #          [ 0.68744134  0.87236687]]"
```

A continuación, se muestran varios ejemplos para **indexar matrices**:

```
import numpy as np

# crear una matriz con rango 2 y dimensiones 3 (filas) x 4 (columnas)
# [[ 1  2  3  4]
#  [ 5  6  7  8]
#  [ 9 10 11 12]]
a = np.array([[1,2,3,4], [5,6,7,8], [9,10,11,12]])

# Utilizamos 'slicing' para extraer un subarray consistente en las filas 0, 1 y las columnas 1,2;
# El resultado 'b' es una matrix 2x2
# [[2 3]
#  [6 7]]
b = a[:2, 1:3]

# Esta submatriz 'b' ('slice of an array') es una vista de los mismos datos,
# con lo cual una modificacion de 'b' tambien cambia la matriz original 'a'
print(a[0, 1]) # muestra "2"
b[0, 0] = 77   # b[0, 0] apunta al mismo dato que a[0, 1]
print(a[0, 1]) # muestra "77"

# extraccion de filas
row_r1 = a[1, :] # segunda fila de 'a' vista como matriz de rango 1 (unidimensional)
row_r2 = a[1:2, :] # segunda fila de 'a' vista como matriz de rango 2 (bidimensional)
print(row_r1, row_r1.shape) # muestra "[5 6 7 8] (4,)"
print(row_r2, row_r2.shape) # muestra "[[5 6 7 8]] (1, 4)"

# extraccion de columnas
col_r1 = a[:, 0] # primera columna de 'a' vista como matriz de rango 1 (unidimensional)
col_r2 = a[:, 0:1] # primera columna de 'a' vista como matriz de rango 2 (bidimensional)
print(col_r1, col_r1.shape) # muestra "[ 1  5  9] (3,)"
print(col_r2, col_r2.shape) # muestra "[[ 1]
#                               [ 5]
#                               [ 9]] (3, 1)"
```

Adicionalmente, pueden realizarse operaciones de indexación mediante condicionales booleanas (Verdadero o Falso):

```
import numpy as np

a = np.array([[1,2], [3, 4], [5, 6]])

bool_idx = (a > 2) # encuentra los elementos que son mayores que 2;
                  # devuelve un numpy array de Booleanos del mismo tamaño (shape)
                  # donde cada elemento de bool_idx indica si el elemento es > 2

print(bool_idx)    # muestra "[[False False]
                  #           [ True  True]
                  #           [ True  True]]"

# Utilizamos el array de booleanos para construir un array de rango 1
# con los elementos que tengan el valor True en bool_idx
print(a[bool_idx]) # muestra "[3 4 5 6]"

# Se puede realizar de manera compacta
print(a[a > 2])    # muestra "[3 4 5 6]"
```

4.1.2 Datatypes

En la creación de *Array*, *Numpy* puede determinar automáticamente el tipo de datos que se contienen en la matriz o se puede indicar de manera explícita el tipo de datos que queremos utilizar. Puede consultar la documentación oficial en <https://docs.scipy.org/doc/numpy/reference/arrays.dtypes.html>

A continuación, se muestran ejemplos para la creación de arrays con **datatypes** implícitos y explícitos:

```
import numpy as np

x = np.array([1, 2]) # numpy selecciona automaticamente datatype
print(x.dtype)      # muestra "int64"

x = np.array([1.0, 2.0]) # numpy selecciona automaticamente datatype
print(x.dtype)          # muestra "float64"

x = np.array([1, 2], dtype=np.int64) # forzar un tipo concreto de datatype
print(x.dtype)                    # muestra "int64"
```

4.1.3 Operaciones

Numpy proporciona funciones matemáticas básicas para operar sobre los elementos de *Array*. Se pueden utilizar las funciones existentes `np.add`, `np.subtract`, `np.multiply`, `np.divide`, ...; y los operadores `+`, `-`, `*` y `/`. A continuación, se muestran varios ejemplos:

```
import numpy as np

#crear dos arrays para operaciones de ejemplo
x = np.array([[1,2],[3,4]], dtype=np.float64)
y = np.array([[5,6],[7,8]], dtype=np.float64)

# Suma elemento a elemento; ambas producen el mismo resultado
# [[ 6.0  8.0]
#  [10.0 12.0]]
print(x + y)
print(np.add(x, y))

# Resta elemento a elemento; ambas producen el mismo resultado
# [[-4.0 -4.0]
#  [-4.0 -4.0]]
print(x - y)
print(np.subtract(x, y))

# Producto elemento a elemento; ambas producen el mismo resultado
# [[ 5.0 12.0]
#  [21.0 32.0]]
print(x * y)
print(np.multiply(x, y))

# Division elemento a elemento; ambas producen el mismo resultado
# [[ 0.2      0.33333333]
#  [ 0.42857143  0.5       ]]
print(x / y)
print(np.divide(x, y))

# Raiz cuadrada elemento a elemento; produce el resultado
# [[ 1.      1.41421356]
#  [ 1.73205081  2.       ]]
print(np.sqrt(x))

# Transpuesta de una matriz
print(x)      # muestra "[[1 2]
               #          [3 4]]"
print(x.T)    # muestra "[[1 3]
               #          [2 4]]"

# Observe que no produce ningun resultado
# tomar la transpuesta de una matriz unidimensional (vector)
v = np.array([1,2,3])
print(v)      # Prints "[1 2 3]"
print(v.T)    # Prints "[1 2 3]"
```

Observe que al contrario que MATLAB, `*` es una multiplicación elemento a elemento, y no se corresponde con una multiplicación de matrices (ver [Artículo en Wikipedia](#)). En su lugar, Numpy proporciona la función `dot`. A continuación, se muestran varios ejemplos:

```
import numpy as np

x = np.array([[1,2],[3,4]]) #array 1 2x2
y = np.array([[5,6],[7,8]]) #array 2 2x2

v = np.array([9,10])        #array 3 1x2
w = np.array([11, 12])      #array 4 1x2

# ejemplos de multiplicacion escalar
# matrices unidimensionales; ambas resultan en el valor 219
print(v.dot(w))
print(np.dot(v, w))

# entre matriz y vector; ambas resultan un array de rango 1 [29 67]
print(x.dot(v))
print(np.dot(x, v))

# matrices bidimensionales; ambas resultan en un array de rango 2
# [[19 22]
#  [43 50]]
print(x.dot(y))
print(np.dot(x, y))
```

Numpy también proporciona varias funciones útiles para realizar cálculos en *Arrays*, como por ejemplo la función `sum` que se muestra a continuación:

```
import numpy as np

x = np.array([[1,2],[3,4]])

print(np.sum(x))          # calcula la suma de todos los elementos; muestra "10"
print(np.sum(x, axis=0))  # calcula la suma de cada columna; muestra "[4 6]"
print(np.sum(x, axis=1))  # calcula la suma de cada fila; muestra "[3 7]"
```

La documentación oficial de las funciones disponibles en el paquete Numpy puede consultarse en la siguiente página web <https://docs.scipy.org/doc/numpy/reference/routines.array-manipulation.html>

4.2 Matplotlib

Matplotlib es una librería para mostrar datos. En esta sección introduciremos el módulo `matplotlib.pyplot`, que proporciona un conjunto de funciones similares a MATLAB. En este documento se emplea la versión 3.0.3 para los ejemplos mostrados.

Existen varias maneras para importar la funcionalidad de Matplotlib en su fichero de código. A continuación se muestran varios ejemplos

```
#Opcion 1 -
import matplotlib.pyplot
matplotlib.pyplot.X           #acceso a la funcion/objeto 'X' de matplotlib.pyplot

#Opcion 2 (aconsejado)
import matplotlib.pyplot as plt   #importar matplotlib.pyplot con el nombre corto 'plt'
plt.X                           #acceso a la funcion/objeto 'X' de numpy

#Opcion 3 (desaconsejado)
from matplotlib.pyplot import *   #importar todas las funciones/objetos
X                                #ya no necesitamos usar matplotlib.pyplot.X o plt.X para acceder
```

4.2.1 Plot

La función más importante de `matplotlib.pyplot` es `plot` que permite mostrar datos 2D. A continuación, se muestra un ejemplo (puede ver el resultado en la Figura 15):

```
#Ejemplo 1
import numpy as np           #importar libreria numpy
import matplotlib.pyplot as plt #importar libreria matplotlib.pyplot

# calculo de datos 2D - funcion sen(x)
x = np.arange(0, 3 * np.pi, 0.1) # coordenadas x para los puntos de la funcion sen()
y = np.sin(x) # calculo de la funcion sen()

# Mostar puntos usando matplotlib
plt.plot(x, y)
plt.show() # debe llamar plt.show() para que se muestre la ventana
```

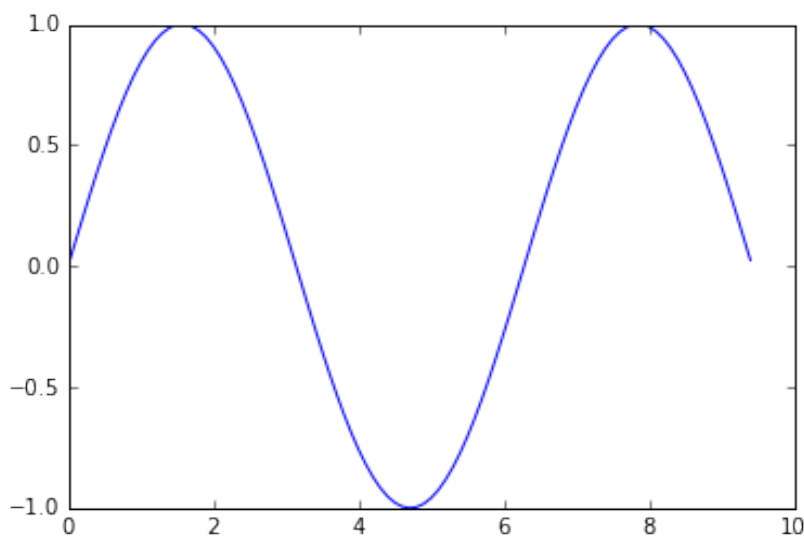


Fig. 15: Resultado de la ejecución del Ejemplo 1

El siguiente ejemplo permite mostrar varios datos simultáneamente con título, leyenda y etiquetas en los ejes (puede ver el resultado en la Figura 16):

```
#Ejemplo 2
import numpy as np
import matplotlib.pyplot as plt

# calculo de datos 2D - funcion sen(x) y cos(x)
x = np.arange(0, 3 * np.pi, 0.1)
y_sin = np.sin(x)
y_cos = np.cos(x)

# mostrar datos usando matplotlib
plt.plot(x, y_sin)
plt.plot(x, y_cos)

# incluir etiqueta para ejes X e Y
plt.xlabel('x axis label')
plt.ylabel('y axis label')

# incluir titulo de figura
plt.title('Sine and Cosine')

# incluir etiqueta/leyenda para los datos pintados
plt.legend(['Sine', 'Cosine'])

# activar ventana
plt.show()
```

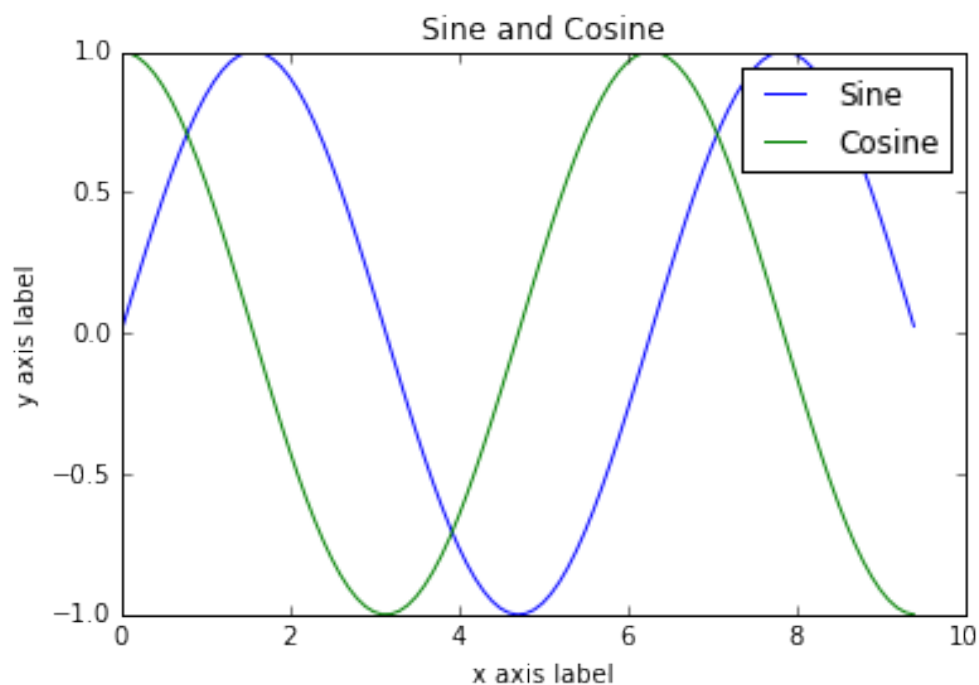


Fig. 16: Resultado de la ejecución del Ejemplo 2

4.2.2 SubPlot

Para mostrar varios datos en una misma ventana, utilizaremos el comando `subplot` de la siguiente manera (puede ver el resultado en la Figura 17):

```
#Ejemplo 3
import numpy as np
import matplotlib.pyplot as plt

# calculo de datos 2D - funcion sen(x) y cos(x)
x = np.arange(0, 3 * np.pi, 0.1)
y_sin = np.sin(x)
y_cos = np.cos(x)

# configurar un subplot con 2 filas y 1 columna ,
# y establecer como activo el primer subplot.
plt.subplot(2, 1, 1)

# primera subventana
plt.plot(x, y_sin) # datos
plt.title('Sine')  # titulo

# segunda subventana
plt.subplot(2, 1, 2) #activamos la posicion a pintar
plt.plot(x, y_cos) # datos
plt.title('Cosine') # titulo

# mostrar la figura
plt.show()
```

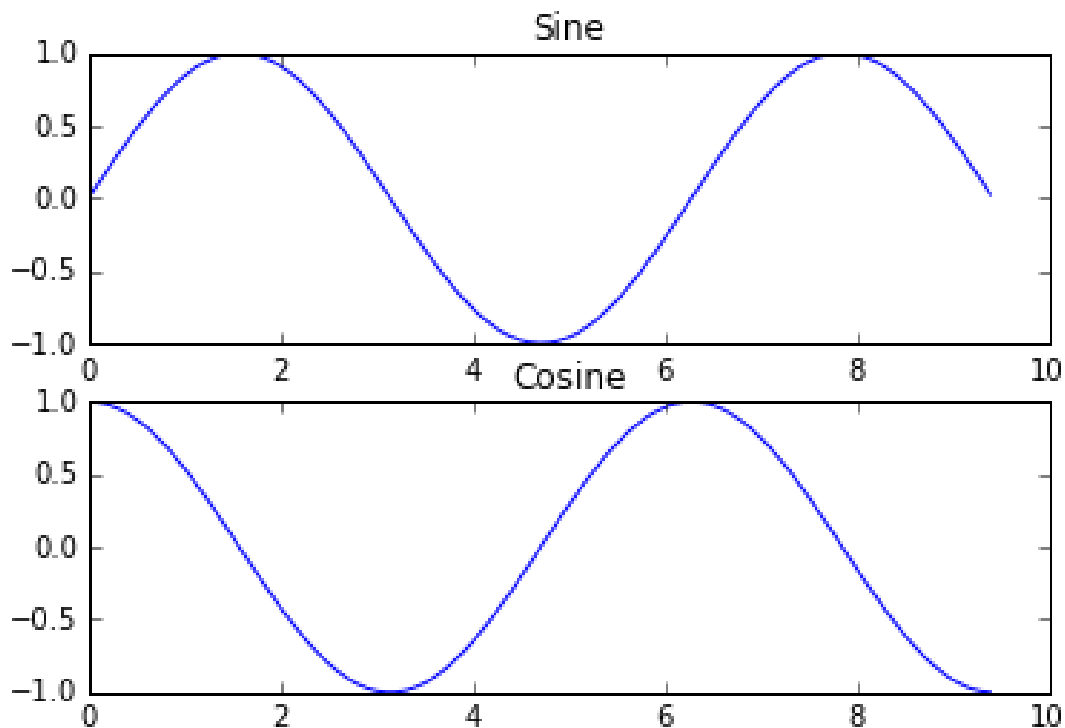


Fig. 17: Resultado de la ejecución del Ejemplo 3

4.2.3 imshow

Para mostrar imágenes, se dispone del comando `imshow` para que el se muestra el siguiente ejemplo:

```
import matplotlib.pyplot as plt
from matplotlib.pyplot import imread

# cargar imagen RGB+alpha y mostrar imagen con mapas de colores RGB (por defecto)
img=imread('https://matplotlib.org/_images/logo2.png')
print(img.dtype, img.shape) # muestra "float32 (130, 542,4)"

plt.figure(1)
imgplot = plt.imshow(img)
plt.show(block=False)

# cargar imagen y mostrar imagen con mapas de colores en escala de grises
img=imread('https://matplotlib.org/_static/stinkbug.png')
print(img.dtype, img.shape) # muestra "float32 (375, 500)"

plt.figure(2)
imgplot = plt.imshow(img, cmap="gray")
plt.show(block=False)

# block execution so the program does not finish and we can see the images
input("Presione cualquier tecla para terminar el ejemplo...")
```

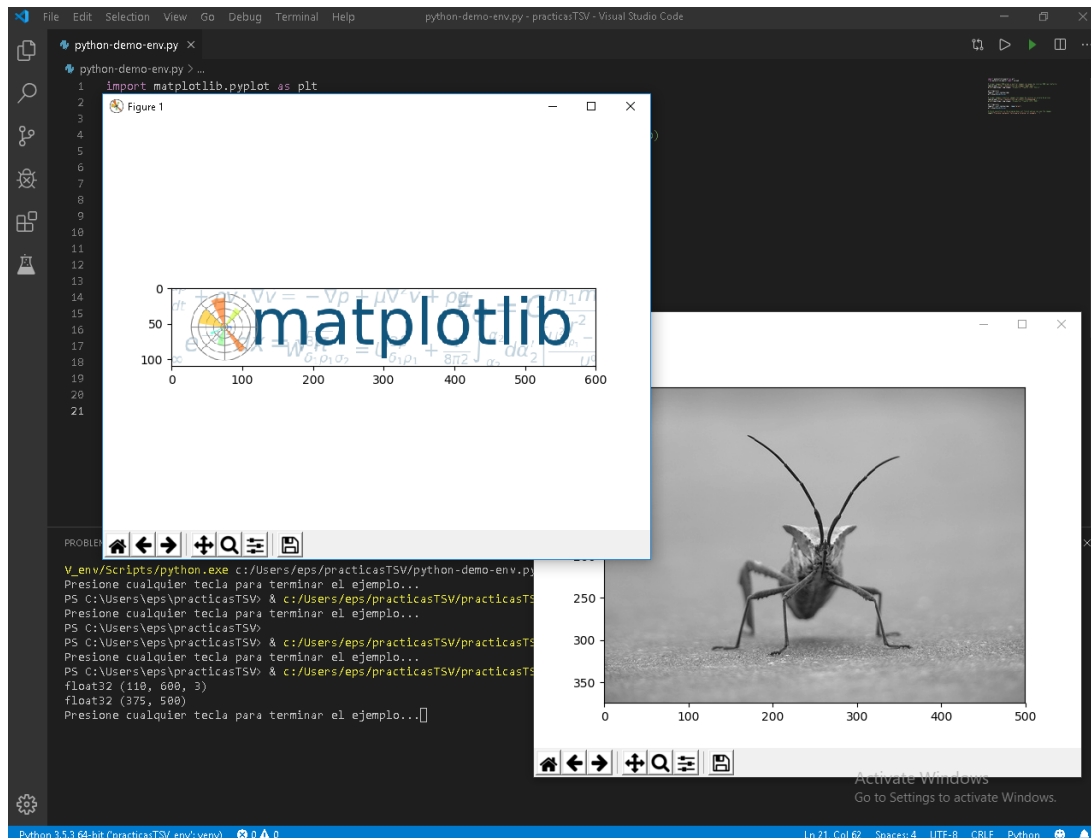


Fig. 18: Resultado de la ejecución del programa de ejemplo para `imshow`

4.3 SciPy

SciPy es una librería que utiliza Numpy para proporcionar una colección algoritmos matemáticos y utilidades que permiten un rápido desarrollo. La documentación oficial de las funciones disponibles en el paquete SciPy puede consultarse en la siguiente página web <https://docs.scipy.org/doc/scipy/reference/>

En esta sección vamos a ver diversos aspectos relacionados con el contenido del curso. La versión que se ha utilizado es la 1.3.0.

4.3.1 Convolución y filtrado de imágenes

SciPy proporciona varias funciones para procesamiento de señales multidimensionales (e.g. imágenes) en su módulo `scipy.ndimage`.

Primero analizamos la **convolución de señales** mediante el método `scipy.ndimage.convolve`. Recuerde la equivalencia de esta operación con el filtrado, si el núcleo aplicado es simétrico en todas las posibles direcciones.

A continuación, se muestra un ejemplo para filtrar una matriz bidimensional (i.e. imagen) utilizando un kernel para detección de bordes. Puede consultar otros kernels en [https://en.wikipedia.org/wiki/Kernel_\(image_processing\)](https://en.wikipedia.org/wiki/Kernel_(image_processing)).

```
import numpy as np
import matplotlib.pyplot as plt
import scipy.ndimage as nd

# matriz/senyal/imagen a filtrar
im = np.zeros((128, 128))
im[32:-32, 32:-32] = 1
im[45:-45, 45:-45] = 2
print(im, im.shape)

# kernel para deteccion de bordes
k = np.array([[0,-1,0],[-1,4,-1],[0,-1,0]])
print(k, k.shape)

# aplicamos el kernel
res = nd.convolve(im, k, mode='constant', cval=0.0)
print(res, res.shape)

# visualiza resultados graficamente
plt.figure(1)
plt.imshow(im, cmap='gray')
plt.show(block=False)

plt.figure(2)
plt.imshow(res, cmap='gray')
plt.show(block=False)

input("Presione cualquier tecla para terminar el ejemplo...")
```

Adicionalmente, SciPy proporciona varias opciones para el **filtrado de imágenes** dentro de la funcionalidad de su módulo `scipy.ndimage`. En el siguiente ejemplo se exploran los filtrados Gaussiano y de Sobel:

```
import numpy as np
import scipy.ndimage as nd
import matplotlib.pyplot as plt

im = np.zeros((256, 256))
im[64:-64, 64:-64] = 1
im[90:-90, 90:-90] = 2

# suavizado con un nucleo Gaussiano
sigma = 8
img = nd.gaussian_filter(im, sigma=sigma)
```

```

# deteccion de bordes con sobel
sx = nd.sobel(im, axis = 0, mode = 'constant') #bordes horizontal
sy = nd.sobel(im, axis = 1, mode = 'constant') #bordes vertical
sob = np.hypot(sx, sy) #modulo de ambos bordes

plt.subplot(3,2,1)
plt.imshow(im, cmap='gray') # mostrar imagen original
plt.title('Imagen Original')

plt.subplot(3,2,2)
plt.imshow(img, cmap='gray') # mostrar imagen suavizada
plt.title('Suavizado Gaussiano')

plt.subplot(3,2,3)
plt.imshow(sx, cmap='gray') # mostrar bordes horizontal
plt.title('SobelX - Bordes horizontal')

plt.subplot(3,2,4)
plt.imshow(sy, cmap='gray') # mostrar bordes vertical
plt.title('SobelY - Bordes vertical')

plt.subplot(3,2,5)
plt.imshow(sob, cmap='gray') # mostrar bordes horizontal y vertical
plt.title('Bordes horizontal y vertical')
plt.show(block=False)

input("Presione cualquier tecla para terminar el ejemplo...")

```

Puede consultar los filtros disponibles en <https://scipy.github.io/devdocs/ndimage.html#module-scipy.ndimage>.

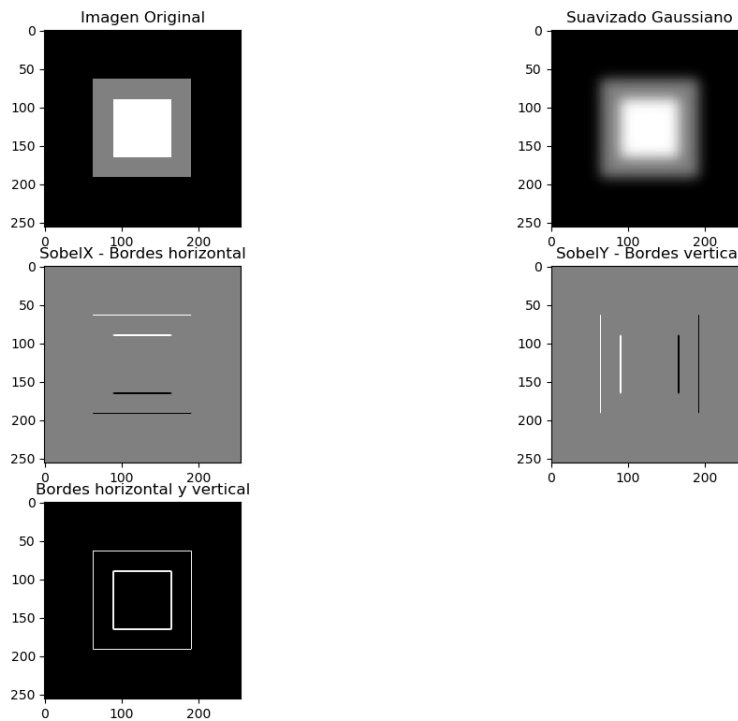


Fig. 19: Resultado de la ejecución de los filtros Gaussiano y de Sobel disponibles en el módulo `scipy.ndimage`

4.3.2 Cálculo de distancias entre puntos

SciPy contiene múltiples funciones para calcular distancias entre puntos en el módulo `scipy.spatial.distance`.

A continuación, se muestran un ejemplo para calcular distancias entre puntos previamente definidos mediante el método `pdist`:

```
# Ejemplo 1
# Calculo de distancias entre todos los puntos con pdist
import numpy as np
from scipy.spatial.distance import pdist

# crea un array donde cada fila son las coordenadas de un punto en un espacio bidimensional 2D
# [[0 1]
#  [1 0]
#  [2 0]]
x = np.array([[0, 1], [1, 0], [2, 0]])
print(x)

# calculo de la distancia Euclidea entre todas las filas de 'x'
# d[i, j] es la distancia euclidea entre el punto i x[i, :] y el punto j x[j, :],
# y 'd' es el siguiente array:
# [[ 0.          1.41421356  2.23606798]
#  [ 1.41421356  0.          1.          ]
#  [ 2.23606798  1.          0.          ]]
d = squareform(pdist(x, 'euclidean'))
print(d)
```

A continuación, se muestran un ejemplo para calcular distancias entre conjuntos de puntos previamente definidos mediante el método `cdist`:

```
# Ejemplo 2
# Calculo de distancias entre todos los pares de puntos de dos conjuntos con cdist
import numpy as np
from scipy.spatial.distance import cdist, squareform

# crea un array donde cada fila son las coordenadas de un punto en un espacio bidimensional 2D
# [[0 1]
#  [1 0]
#  [2 0]]
x = np.array([[0, 1], [1, 0], [2, 0]])
print(x)

# crea un array donde cada fila son las coordenadas de un punto en un espacio bidimensional 2D
# [[0 1]
#  [1 1]
#  [2 3]]
y = np.array([[0, 1], [1, 1], [2, 3]])
print(x)

# calculo de la distancia Coseno entre todas las filas de 'x'
# d[i, j] es la distancia Coseno entre el punto i x[i, :] y el punto j y[j, :],
# y 'd' es el siguiente array:
# [[ 0.          0.29289322  0.16794971]
#  [ 1.          0.29289322  0.4452998 ]
#  [ 1.          0.29289322  0.4452998 ]]
d = cdist(x, y, 'cosine')
print(d)
```

Los métodos `pdist` y `cdist` soportan varios tipos distintos de distancias además de 'euclidean' y 'cosine'. Puede consultar todas las distancias disponibles en las siguientes páginas web [Ayuda función pdist](#) y [Ayuda función cdist](#)

4.4 Scikit-image

Scikit-image es una librería específicamente diseñada para el manejo y tratamiento de imágenes en Python. Notése que la librería SciPy tiene un carácter más amplio (orientada al campo de las matemáticas, ciencia y ingeniería) y por ello proporciona una menor funcionalidad respecto al tratamiento de imágenes. Este documento se ha preparado con la versión 0.15.0 de scikit-image.

La documentación oficial está disponible en <https://scikit-image.org/docs/stable/>. En esta sección mostraremos aspectos básicos para su uso durante este curso.

4.4.1 Lectura de imágenes

Para la lectura de imágenes se utiliza una combinación de los módulos `skimage.io` y `matplotlib.pyplot` como se muestra a continuación:

```
import matplotlib.pyplot as plt
import numpy as np
from skimage import io

# lectura de imagen desde URL
img=io.imread('https://matplotlib.org/_static/stinkbug.png')
print(img.dtype, img.shape) # muestra uint8 (375, 500)

# mostrar imagen
plt.imshow(img, cmap='gray')
plt.show()
```

Si depura este código, puede observar que la imagen tiene tipo `uint8` cuyos valores están en el rango `[0, 255]`. Si por el contrario quiere operar en el rango `[0, 1]`, debe convertir la imagen leída con `img_as_float32` como se muestra en el siguiente código:

```
import matplotlib.pyplot as plt
import numpy as np
from skimage import io, img_as_float32

# lectura de imagen desde URL
img=io.imread('https://matplotlib.org/_static/stinkbug.png')
print(img.dtype, img.shape) # muestra uint8 (375, 500)

floatImage = img_as_float32(image)
print(floatImage.dtype, floatImage.shape) # muestra float32 (375, 500)

# mostrar imagen
plt.imshow(floatImage, cmap='gray')
plt.show()
```

4.4.2 Escala de grises VS Color

Las imágenes en color se definen principalmente como un conjunto de matrices bidimensionales, cada matriz con valores representativos de la intensidad de la imagen. Por ejemplo, las imágenes a color RGB están compuestas de tres matrices independientes (también llamadas *canales*) para representar los colores rojo (*red*), verde (*green*) y azul (*blue*). Por otro lado, las imágenes en escala de grises están formadas por una sola matriz que representa el nivel de gris o intensidad (*gray*).

En Python, podemos convertir imágenes de color a escala de grises con el método `rgb2gray`. También podemos utilizar la indexación descrita en la sección 4.1.1 para acceder a los datos de cada una de las matrices de una imagen RGB. Para mostrar imágenes seguiremos utilizando Matplotlib y su función `matplotlib.pyplot.imshow`. El siguiente código muestra un ejemplo de toda la funcionalidad descrita:

```

import numpy as np
from skimage import io, color
import matplotlib.pyplot as plt

# lectura de imagen RGB desde URL
rgbImage = io.imread('http://www.jeffreysward.com/image/tp02aw600.gif')
print(rgbImage.dtype, rgbImage.shape)

# conversion a escala de grises
grayImage = color.rgb2gray(rgbImage)
print(grayImage.dtype, grayImage.shape)

# mostrar imagen original y convertida
plt.figure(1)
plt.imshow(rgbImage)
plt.figure(2)
plt.imshow(grayImage, cmap='gray')
plt.show(block=False)

# extraer canales de color
redChannel = rgbImage[:, :, 0]    # canal rojo
greenChannel = rgbImage[:, :, 1]  # canal verde
blueChannel = rgbImage[:, :, 2]   # canal azul

# mostrar los canales de color
plt.figure(3)
plt.imshow(redChannel, cmap='gray'); plt.title('Rojo')
plt.figure(4)
plt.imshow(greenChannel, cmap='gray'); plt.title('Verde')
plt.figure(5)
plt.imshow(blueChannel, cmap='gray'); plt.title('Azul')
plt.show()

```

En el siguiente ejemplo vamos a mostrar como se puede generar una imagen RGB a partir de tres matrices utilizando la función `np.stack` de la librería Numpy:

```

import numpy as np
from skimage import io, color
import matplotlib.pyplot as plt

# lectura de imagen RGB desde URL
rgbImage = io.imread('http://www.jeffreysward.com/image/tp02aw600.gif')
print(rgbImage.dtype, rgbImage.shape)
(m, n, o) = rgbImage.shape

# extraer canales de color
redChannel = rgbImage[:, :, 0]    # canal rojo
greenChannel = rgbImage[:, :, 1]  # canal verde
blueChannel = rgbImage[:, :, 2]   # canal azul

# crear una imagen vacia (de niveles de gris)
allBlack = np.zeros((m, n), dtype=np.uint8)

# crear version de color para cada canal extraido
justRed = np.stack((redChannel, allBlack, allBlack), axis=2)
justGreen = np.stack((allBlack, greenChannel, allBlack), axis=2)
justBlue = np.stack((allBlack, allBlack, blueChannel), axis=2)

# Recombinar los canales extraidos para recrear la imagen original RGB
recombinedRGBImage = np.stack((redChannel, greenChannel, blueChannel), axis=2)
plt.imshow(recombinedRGBImage)
plt.show()

```

4.4.3 Convolución y filtrado

Para calcular la convolución y filtrado (si el kernel es isotrópico), utilizaremos la misma funcionalidad de SciPy que hemos visto en la sección 4.3.1. El siguiente ejemplo muestra como aplicar un filtro de media a una imagen.

```
import numpy as np
import matplotlib.pyplot as plt
from skimage import color, data
from scipy.signal import convolve2d as conv2
from scipy.ndimage import convolve

# lectura de imagen desde el modulo 'data'
astro = color.rgb2gray(data.astronaut())

# kernel 5x5 para el filtro de media
k = np.ones((5, 5)) / 25

# aplicamos el kernel (ambas opciones son equivalentes)
astro_ = conv2(astro, k, 'same') # opcion 1
astro_ = convolve(astro, k, mode='constant') # opcion 2

fig, ax = plt.subplots(nrows=1, ncols=2, figsize=(8, 5),
                       sharex=True, sharey=True)

plt.gray()

ax[0].imshow(astro, cmap='gray')
ax[0].axis('off')
ax[0].set_title('Imagen Astronaut del modulo skimage.Data')

ax[1].imshow(astro_)
ax[1].axis('off')
ax[1].set_title('Suavizado con un filtro de media 5x5')

fig.tight_layout()

plt.show()
```

Imagen Astronaut del módulo skimage.Data



Suavizado con un filtro de media 5x5



Fig. 20: Resultado de la ejecución del filtro de media disponibles en el módulo `scipy.ndimage` y `skimage`

5 Formato ficheros de las prácticas

En cada práctica, se le proporcionarán archivos como el mostrado en la Figura 21, en los cuales se le pedirá que complete las funciones que aparecen vacías. Para verificar el correcto funcionamiento de su desarrollo, simplemente deberá ejecutar el archivo correspondiente.

Tenga en cuenta las líneas de código que siguen a la sentencia `if __name__ == "__main__"` (línea 41). Esta sentencia nos permite identificar el archivo principal que ejecuta el código del proyecto. Además, es esencial para evitar que el código se ejecute automáticamente cuando el archivo es importado desde otro script en Python. Sin este `if`, el código se ejecutaría cada vez que se importe el archivo, incluso si solo se utilizan las funciones. Estas líneas tras `if __name__ == "__main__"` contendrán llamadas a funciones que verifiquen el funcionamiento de las funciones que ha implementado. En la Figura 22 puede ver la solución correcta para este fichero mostrado en la Figura 21.

```
p0 > misc > test_formatopracticas.py > resta
1  # Tratamiento de Señales Visuales/Introducción a la Visión Artificial @ EPS-UAM
2  # Practica 0: Introducción a Python
3
4  # AUTOR1: APELLIDO1 APELLIDO1, NOMBRE1
5  # AUTOR2: APELLIDO2 APELLIDO2, NOMBRE2
6  # PAREJA/TURNO: NUMERO_PAREJA/NUMERO_TURNO
7  import numpy as np
8
9  def suma(x,y):
10     """
11     # Esta funcion implementa la operacion de suma de arrays
12     #
13     # Argumentos de entrada:
14     #   x: numpy array de tamaño [imagen_height, imagen_width].
15     #   y: numpy array de tamaño [imagen_height, imagen_width].
16     #
17     # Devuelve:
18     #   output: numpy array de tamaño [imagen_height, imagen_width].
19     """
20
21
22
23     return output
24
25  def resta(x,y):
26     """
27     # Esta funcion implementa la operacion de resta de arrays
28     #
29     # Argumentos de entrada:
30     #   x: numpy array de tamaño [imagen_height, imagen_width].
31     #   y: numpy array de tamaño [imagen_height, imagen_width].
32     #
33     # Devuelve:
34     #   output: numpy array de tamaño [imagen_height, imagen_width].
35     """
36
37
38     return output
39
40  if __name__ == "__main__":
41      print("Practica 0 - Tarea X \n")
42
43      x = np . random . random (( 2 , 2 ) )
44      y = np . random . random (( 2 , 2 ) )
45      print("Ejecución funcion suma = " + str(suma(x,y)))
46      print("Ejecución funcion resta = " + str(resta(x,y)))
```

Fig. 21: Formato de ficheros de prácticas.

6 Referencias

Para la elaboración de este tutorial se han utilizado diversas fuentes listadas a continuación:

- [Documentación oficial de Python 3.6](#)
- [Documentación oficial de Scipy 1.3.0](#)
- [Documentación oficial de SciKit-image](#)
- [Python Numpy Tutorial, CS231n Convolutional Neural Networks for Visual Recognition, Stanford](#)
- [Python Primer, CSCI 1430: Introduction to Computer Vision, Brown University](#)
- [Tutorial Python-SciPy at tutorialspoint.com](#)
- [Tutorial Python en español](#)

```
p0 > misc > test_formatopracticas.py > ...
1  # Tratamiento de Señales Visuales/Introducción a la Visión Artificial @ EPS-UAM
2  # Practica 0: Introducción a Python
3
4  # AUTOR1: APELLIDO1 APELLIDO1, NOMBRE1
5  # AUTOR2: APELLIDO2 APELLIDO2, NOMBRE2
6  # PAREJA/TURNO: NUMERO_PAREJA/NUMERO_TURNO
7  import numpy as np
8
9  def suma(x,y):
10     """
11     # Esta funcion implementa la operacion de suma de arrays
12     #
13     # Argumentos de entrada:
14     #   x: numpy array de tamaño [imagen_height, imagen_width].
15     #   y: numpy array de tamaño [imagen_height, imagen_width].
16     #
17     # Devuelve:
18     #   output: numpy array de tamaño [imagen_height, imagen_width].
19     """
20
21     output = x+y
22
23     return output
24
25  def resta(x,y):
26     """
27     # Esta funcion implementa la operacion de resta de arrays
28     #
29     # Argumentos de entrada:
30     #   x: numpy array de tamaño [imagen_height, imagen_width].
31     #   y: numpy array de tamaño [imagen_height, imagen_width].
32     #
33     # Devuelve:
34     #   output: numpy array de tamaño [imagen_height, imagen_width].
35     """
36
37     output = x-y
38
39     return output
40
41  if __name__ == "__main__":
42     print("Practica 0 - Tarea X \n")
43
44     x = np . random . random (( 2 , 2 ) )
45     y = np . random . random (( 2 , 2 ) )
46     print("Ejecución funcion suma = " + str(suma(x,y)))
47     print("Ejecución funcion resta = " + str(resta(x,y)))
```

Fig. 22: Formato de ficheros de prácticas (solución)